MessManager - Bachelor Mess Expense & Meal System

Hasan Imam

ID# 22203032

A Practicum in the Partial Fulfillment of the Requirements

for the Award of Bachelor of Computer Science and Engineering (BCSE)



Department of Computer Science and Engineering
IUBAT School of Computer Science and Engineering
IUBAT—International University of Business Agriculture and Technology

Spring 2026

MessManager - Bachelor Mess Expense & Meal System

Hasan Imam

A Practicum in the Partial Fulfillment of the Requirements for the Award of Bachelor of Computer Science and Engineering (BCSE)

The practicum has been examined and approved,

_____

Prof. Dr. Utpal Kanti Das
Dean and Chair

_____

Shahinur Alam
Co-supervisor, Coordinator and Assistant Professor

_____

Jannatul Ferdous
Supervisor and Lecturer

Department of Computer Science and Engineering
IUBAT School of Computer Science and Engineering
IUBAT—International University of Business Agriculture and Technology

Spring 2026

Letter of Transmittal

11 May 2026

The Chair

Practicum Defense Committee

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

4 Embankment Drive Road, Sector 10, Uttara Model Town

Dhaka 1230, Bangladesh.

Subject: Letter of Transmittal.

Dear Sir,

With due respect, I am pleased to submit my practicum report titled "MessManager - Bachelor Mess Expense & Meal System," developed as a partial requirement for the Bachelor of Computer Science and Engineering (BCSE) degree at IUBAT. During my three-month practicum at CODETREE, I was involved in the design, development, and implementation of a centralized mess management system aimed at automating meal tracking, bazaar expense logging, and complex monthly bill calculations.

I would like to express my sincere gratitude to the CODETREE team for their professional cooperation and technical support throughout the practicum period.

Yours sincerely,

_____

Hasan Imam

22203032

Organization's Certificate



CERTIFICATE

OF COMPLETION

THIS CERTIFIES THAT

Hasan Imam

Has completed Web Development INTERNSHIP at CODETREE from 15-2-2026 to 22-5-2026
We found him/her sincere, hardworking, dedicated and result oriented. He/she worked well as part of the team during his/her tenure.
We take this opportunity to thank him/her and wish him/her all the best for his/her future endeavours.

CODETREE
WE CODE BETTER LIFE
House 20 (Level 01), Road 13 , Sector 07, Uttara Model Town, Dhaka 1230

Chairman / Managing Director

Student's Declaration

I hereby declare that the practicum report titled "MessManager - Bachelor Mess Expense & Meal System" is my own original work. This report is submitted in partial fulfillment of the requirements for the degree of Bachelor of Computer Science and Engineering (BCSE) at IUBAT—International University of Business Agriculture and Technology. I further declare that this report has not been submitted to any other university or institution for any degree or diploma. All sources of information and data used in this report have been duly acknowledged and cited according to academic standards.

_____

Hasan Imam

22203032

Supervisor's Certification

This is to certify that the practicum report titled "MessManager - Bachelor Mess Expense & Meal System" has been prepared by Hasan Imam, ID#22203032, under my direct supervision and guidance. To the best of my knowledge, the work presented in this report is original and fulfills the requirements for the award of the degree of Bachelor of Computer Science and Engineering (BCSE). The report is now ready for examination and defense.

_____

Jannatul Ferdous
Supervisor and Lecturer
Department of Computer Science and Engineering
IUBAT—International University of Business Agriculture and Technology

Abstract

Managing shared living expenses and meal systems (messes) in Bangladesh is traditionally a manual, error-prone process. This report describes the development of MessManager, a Bachelor Mess Expense & Meal System designed to automate meal tracking, expense management, and billing calculations. Built using Laravel 10 for the backend and React 19 for the frontend, the system provides role-based dashboards for administrators, managers, and members. Key features include a dynamic meal rate calculator based on bazaar expenses, an approval-based deposit workflow, and automated splitting of monthly fixed costs like rent and utilities. The system ensures financial transparency and operational efficiency, replacing paper logs with a secure digital environment. Future work involves integrating character-level OCR to automate bazaar receipt data entry.

Acknowledgments

First, I would like to express my deepest gratitude to Almighty Allah for giving me the strength and patience to complete this practicum successfully.

I am profoundly grateful to my supervisor, Jannatul Ferdous, for providing invaluable guidance, constant encouragement, and technical insights throughout the development of MessManager. I also wish to thank Prof. Dr. Utpal Kanti Das, Dean and Chair of the Department of CSE, and Shahinur Alam, Co-supervisor and Coordinator, for their administrative support and academic mentorship.

Special thanks go to the team at CODETREE for providing me with a professional environment and the resources necessary to implement this project as a Trainee IT Executive. Finally, I am thankful to my family and friends for their continuous support and motivation during my undergraduate studies.

Table of Contents

Letter of Transmittal iii
Organization's Certificate iv
Student's Declaration v
Supervisor's Certification vi
Abstract vii
Acknowledgments viii
List of Figures xiv
List of Tables xvi
Chapter 1. 1
Introduction 1
1.1 Background of the Study 2
1.2 Methodology 2
1.2.1 Primary Sources 2
1.2.2 Secondary Sources 2
1.3 Objectives 3
1.3.1 Broad Objective 3
1.3.2 Specific Objectives 3
1.4 Process Model 3
1.5 Feasibility Study 4
1.5.1 Technical Feasibility 4
1.5.2 Economic Feasibility 5
1.5.3 Operational Feasibility 5
1.6 Structure of the Report 5
Chapter 2. 6
Organizational Overview 6

2.1 Organization Profile 7
2.2 Organization Vision 8
2.3 Organization Mission 8
2.4 Organization Services 8
2.5 Organizational Structure 9
2.6 My Position in this Organization 9
2.7 Address of the Organization 10
Chapter 3. 11
Requirement Engineering 11
3.1 Requirement Elicitation & Analysis 12
3.2 Functional Requirements (FR) 12
3.3 Non-Functional Requirements 13
3.4 Use Case Diagram 14
Chapter 4. 16
System Analysis 16
4.1 Activity Diagrams 17
4.1.1: Admin Activity Diagram 17
4.1.2: Manager Activity Diagram 19
4.1.3: User Activity Diagram 20
4.1.4: Meal Entry Activity Diagram 21
4.1.5: Payment Processing Activity Diagram 22
4.2 Swimlane Diagram 23
4.2.1 Deposit Approval Flow 23
4.2.2: Login & Authentication Swimlane Diagram 23

4.2.3: Mess Creation & Admin Approval Swimlane Diagram.....24

4.2.4: Member Onboarding Swimlane Diagram.....24

4.2.5: Meal Entry & Bill Tracking Swimlane Diagram.....25

4.2.6: Deposit Request & Approval Swimlane Diagram.....25

4.2.7: Expense Management & Monthly Reports Swimlane Diagram 26

4.3 Class Diagram.....27

4.4 Sequence Diagram 28

4.4.1 Monthly Billing Calculation 28

4.5 CRC Cards 30

Chapter 5. 36

Risk Management 36

5.1 Risk Identification.....37

5.2 Risk Analysis 37

5.3 Risk Prioritization 38

5.4 RMMM Plan 38

5.4.1 RMMM for Technical Risks (System Stability).....38

5.4.2 RMMM for Security Risks (Data Protection).....38

5.5 Monitoring & Review Plan.....39

Chapter 6. 40

Project Planning & Scheduling.....40

6.1 Function Point Estimation.....41

6.1.1 Functionality, Input, Output.....41

6.1.2 Identify Complexity 42

6.1.3 Unadjusted Function Point Contribution 43
6.1.4 Calculation of Total Degree of Influence (TDI) 45
6.1.5 Final Calculation 45
6.2 Project Scheduling 46
6.2.1 Gantt Chart Breakdown 46
6.3 Resource & Cost Estimate 47
Chapter 7. 48
System Design & Implementation 48
7.1 System Architecture 49
7.2 Data Flow Diagrams (DFD) 49
7.3 Entity Relationship Diagram (ERD) 56
7.3.1 Database Field Design 57
7.4 Interface Design 61
7.4.1 Home Page 61
7.4.2 Login Page 62
7.4.3 Admin Interface 63
7.4.4 Manager Interface 65
7.4.5 User Interface 68
7.5 API Design Summary 70
7.6 Implementation Notes 71
Chapter 8. 72
Testing 72
8.1 Test Strategy 73
8.2 Test Environment & Data 73

8.3 Test Case Tables 73
8.4 Test Results & Bug Log 74
8.5 Performance & Security Checks 75
Chapter 9. 76
Ethical Considerations & Sustainability 76
9.1 Ethical Considerations 77
9.1.1 Data Privacy and Security 77
9.1.2 Responsibility to Stakeholders 77
9.2 Sustainability Through Software Engineering 78
9.2.1 Paperless Impact (Social & Environmental Sustainability) 78
9.2.2 Economical Sustainability 78
9.3 Professional Responsibility 78
Chapter 10. 79
Conclusion 79
10.1 Conclusion 80
10.2 Limitations 80
10.3 Future Work & Roadmap 81
10.3.1 Character-Level OCR Integration 81
10.3.2 Functional Roadmap 81

List of Figures

Figure 1.1 Agile Process Model....4
Figure 2.1: Organization's Logo....7
Figure 2.2: 'CODETREE' Company Structure....9
Figure 2.3: Organizational Location Map....10
Figure 3.1: Use Case Diagram for MessManager....15
Figure 4.1.1: Activity Diagram for Admin....18
Figure 4.1.2: Activity Diagram for Manager....19
Figure 4.1.3: Activity Diagram for User....20
Figure 4.1.4: Activity Diagram for Meal Entry....21
Figure 4.1.5: Activity Diagram for Payment Processing....22
Figure 4.2.1: Swimlane Diagram for Login & Authentication....23
Figure 4.2.2: Swimlane Diagram for Mess Creation & Admin Approval....24
Figure 4.2.3: Swimlane Diagram for Member Onboarding....24
Figure 4.2.4: Swimlane Diagram for Meal Entry & Bill Tracking....25
Figure 4.2.5: Swimlane Diagram for Deposit Request & Approval....25
Figure 4.2.6: Swimlane Diagram for Expense Management & Monthly Reports....26
Figure 4.3: MessManager Class Diagram....27
Figure 4.4: Billing Calculation Sequence Diagram....29
Figure 6.1: MessManager 12-Week Gantt Chart....46
Figure 7.1 DFD Level 0....50
Figure 7.2 DFD Level 1....51
Figure 7.3 DFD Level 2 (1)....52
Figure 7.4 DFD Level 2 (2)....52
Figure 7.5 DFD Level 2 (3)....53
Figure 7.6 DFD Level 2 (4)....53
Figure 7.7 DFD Level 2 (5)....54
Figure 7.8 DFD Level 2 (6)....54
Figure 7.9 DFD Level 2 (7)....55

Figure 7.10 Entity Relationship Diagram 56
Figure 7.11 Database field design.....57
Figure 7.12 User Table 57
Figure 7.13 Expense Table 58
Figure 7.14 Fixed Costs Table 58
Figure 7.15 Meals Table 59
Figure 7.16 Messes Table 59
Figure 7.17 Mess Request Table.....59
Figure 7.18 User_Deposite_Requests 60
Figure 7.19 User Transaction Table.....60
Figure 7.20 Home Page.....61
Figure 7.21 Login Page.....62
Figure 7.22 Admin Dashboard.....63
Figure 7.23 Mess Group Request.....63
Figure 7.24 Manager Details.....64
Figure 7.25 Create Mess Request 65
Figure 7.26 Manager Dashboard.....66
Figure 7.27 Add New Member 66
Figure 7.28 Add Expense.....67
Figure 7.29 Meals Chart 67
Figure 7.30 Monthly Report 68
Figure 7.31 User Dashboard 68
Figure 7.32 Meal Entry.....69
Figure 7.33 My Bill.....69
Figure 7.34 Balance 70
Figure 10.1 Plagiarism Report 83

List of Tables

Table 3.1 Functional Requirements 12
Table 4.1 CRC of User 30
Table 4.2 CRC of Mess 30
Table 4.3 CRC of MessRequest 31
Table 4.4 CRC of Meal 31
Table 4.5 CRC of Expense 32
Table 4.6 CRC of FixedCost 32
Table 4.7 CRC of UserTransaction 33
Table 4.8 CRC of UserDepositRequest 33
Table 4.9 CRC of AuthService 34
Table 4.10 CRC of MemberManagementService 34
Table 4.11 CRC of BillingCalculator 35
Table 4.12 CRC of BillingCalculator 35
Table 5.1 Risk Identification 37
Table 5.1 Risk Analysis 37
Table 6.1 Functionality, Input, Output 41
Table 6.2: Identify Complexity of Data Function 42
Table 6.3: Identify Complexity of Transaction Function 43
Table 6.4: UFP of Transaction Function 43
Table 6.5: UFP of Data Function 44
Table 6.6: UFP of Data Function 45
Table 7.5: API Design Summary 70
Table 8.1 Authentication and Role Access 74
Table 8.2 Core Functional Flows 74

Chapter 1.

Introduction

Manual management of a shared mess (meal system) often leads to significant challenges, including errors in bazaar cost tracking, inconsistent meal rate calculations, and complex manual splits. To address these issues, MessManager was developed as a unified digital platform to automate and centralize all mess operations. This chapter provides the background, objectives, and foundational methodology utilized in the development of the system.

1.1 Background of the Study

In the context of student housing and shared living in Bangladesh, people frequently struggle to answer questions regarding individual meal consumption, bazaar vs. fixed cost expenditures, and precise balances after deposits. Traditional manual tracking is prone to human error and lack of transparency. The "Background of the Study" provides the justification for automating these processes to ensure a fair and efficient environment for all members.

1.2 Methodology

The development of MessManager was grounded in a thorough research process aimed at understanding the specific operational challenges of shared living. To ensure a robust system, data was gathered from two distinct categories of information sources:

1.2.1 Primary Sources

- **Interviews:** Conducted conversations with mess managers and members to identify daily problems like hard-to-track paper records.
- **Observation:** Watched existing manual processes to see where delays occur, such as manual meal calculations and deposit tracking.
- **User Workshops:** Direct insights were used to define core requirements like role-based access and automated billing logic.

1.2.2 Secondary Sources

- **Technical Documentation:** Reviewing Laravel 10 and React 19 documentation to ensure industry-standard security and coding practices.

- **Literature Review:** Analyzing existing software solutions and management systems to benchmark features and UI design.
- **Web Resources:** Utilizing online repositories and research papers regarding automated billing and system design.

1.3 Objectives

The goals of the proposed system are divided into broad and specific categories.

1.3.1 Broad Objective

The main goal of this project is to create an online MessManager system to automate daily tasks, moving away from paper-based documentation to a paperless, digital environment that ensures transparency.

1.3.2 Specific Objectives

- To create a secure, role-based platform for superAdmin, manager, and user (member) roles.
- To build a robust booking engine for daily meal tracking (Lunch, Dinner, or Off).
- To implement a dynamic billing system that calculates the meal rate based on bazaar expenses and splits fixed costs equally.
- To integrate an approval-based deposit flow that updates user balances in real-time.
- To design user-friendly dashboards for monitoring transactions and monthly bill summaries.

1.4 Process Model

The Agile Process Model was utilized for the development of MessManager. This iterative approach allowed for flexibility, breaking the system into manageable modules such as Appointment, Report, and Admin Dashboards.

Reasons for choosing the Agile Model:

- **Adaptability:** Allowed for quick changes based on feedback during sprints.

- **Early Bug Detection:** Module-wise testing helped identify and fix issues early in the lifecycle.
- **User-Centric Design:** Continuous feedback from the target user group improved overall usability.

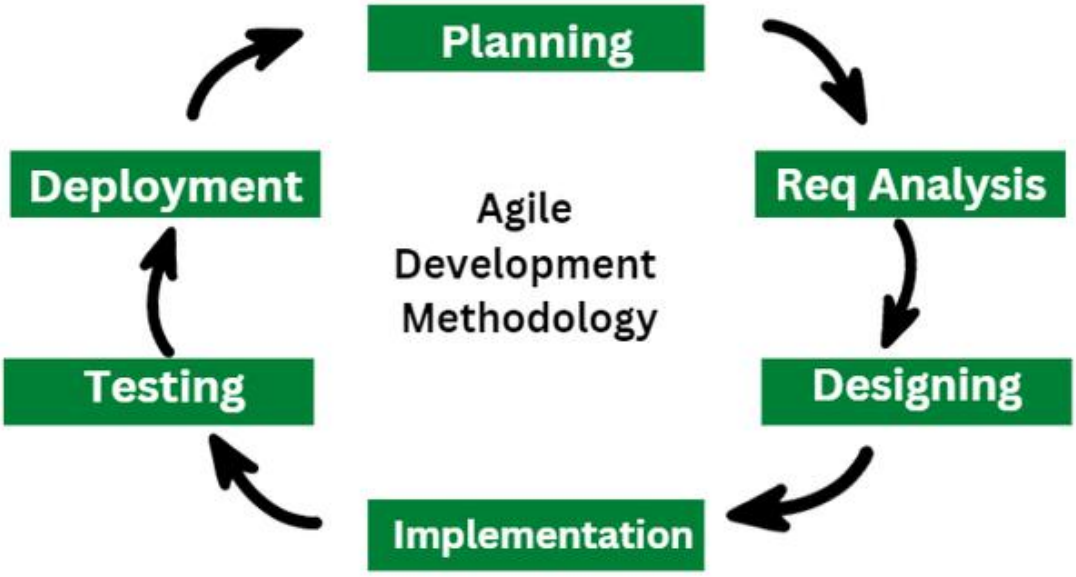


Figure 1.1 Agile Process Model

1.5 Feasibility Study

This study evaluates whether the system can be developed efficiently within the available resources and organizational environment.

1.5.1 Technical Feasibility

The project is technically feasible as it utilizes the modern and mature Laravel/React stack. (Otwell, 2023) Laravel provides strong security features like CSRF protection and Bcrypt hashing, while MySQL ensures reliable relational data management.

1.5.2 Economic Feasibility

MessManager is economically feasible because it utilizes open-source technologies (PHP, Laravel, React, MySQL), significantly reducing software licensing costs. By automating billing and expenses, it reduces human error and administrative overhead.

1.5.3 Operational Feasibility

The system is designed with a user-centric approach, providing distinct and intuitive dashboards for different roles. It aligns with real-world mess operations, making it easy for members and managers to adopt without extensive training.

1.6 Structure of the Report

The report is structured into ten chapters, beginning with this introduction, followed by chapters that delve into the Organizational Overview, Requirement Engineering, System Analysis, Risk Management, Project Scheduling, System Design, Testing, Ethical Considerations, and a final Conclusion.

Chapter 2.

Organizational Overview

2.1 Organization Profile

CODETREE is a software and IT solutions company based in Bangladesh that has been in business since 2017. It has offices in Dhaka and Ontario, Canada. The company focuses on providing a variety of digital services, such as creating custom software, designing websites and web apps, developing mobile apps, graphic design, image post-production, digital marketing, and bulk SMS services. CODETREE also makes specialized systems like school and college management solutions, inventory and POS systems, and platforms for managing donations. The company is known for making solutions that are easy to use, efficient, and customizable. It stresses clean coding, responsive designs, and quick delivery, and it also makes sure that customers can get help 24/7. Over the years, CODETREE has earned the trust of its clients by providing innovative, dependable, and affordable digital solutions, working with industry leaders like SSLCOMMERZ, and serving as an official IT partner for a number of local and international events.



Figure 2.1: Organization’s Logo

2.2 Organization Vision

Vision: “We code better life.” This concise, aspirational statement reflects CODETREE’s commitment to enhancing everyday experiences through smart, efficient, and life-enriching digital solutions.

2.3 Organization Mission

CODETREE is driven by a bold mission: to unite the diverse expertise of designers, developers, and entrepreneurs under one roof—one “tree”—to deliver the best-quality digital outcomes tailored to client needs.

2.4 Organization Services

CODETREE offers a comprehensive suite of services, catering to a wide range of digital and technological needs. Key services include:

- **Web Development & Design:** Creating dynamic, responsive websites.
- **App Development:** Building custom mobile applications.
- **Graphic Design:** Crafting visual identities and marketing materials.
- **Digital Marketing:** Implementing online advertising and promotion strategies.
- **Bulk SMS Services:** Mass messaging for customer outreach.
- **Cloud Hosting:** Reliable infrastructure for hosting client solutions.

Additionally, they provide specialized software solutions such as:

- **Point-of-Sale (POS) & Inventory Management**.
- **School & College Management Systems**.
- **Donation Management Platforms**.

2.5 Organizational Structure

The company follows a hierarchical model that facilitates collaboration between design, development, and management teams.

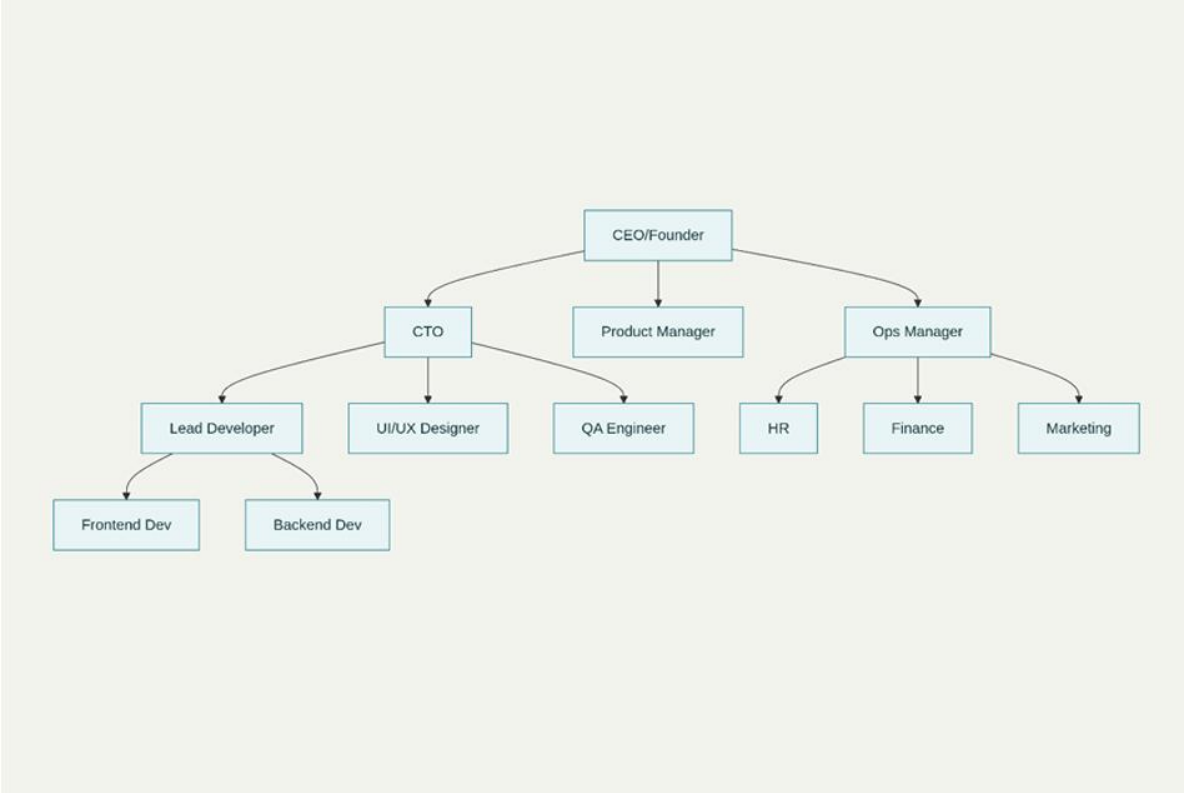


Figure 2.2: ‘CODETREE’ Company Structure

2.6 My Position in this Organization

During my practicum at CODETREE, I served in the **“**TRAINEE IT EXECUTIVE PROGRAM**”**. My primary responsibilities included assisting in the design and development

of the MessManager system for CODETREE, implementing both frontend and backend features, testing system functionalities, and collaborating with the design and development teams to ensure quality and timely delivery.

2.7 Address of the Organization

CODETREE: House 73/F, Road 12/B, Sector 10, Uttara, Dhaka 1230, Bangladesh.

Contact Information:

- **Phone:** 01710070606.
- **Email:** codetreebd@gmail.com.
- **Working Hours:** 09:00 AM – 06:00 PM.
- https://www.google.com/maps/place/CODETREE/

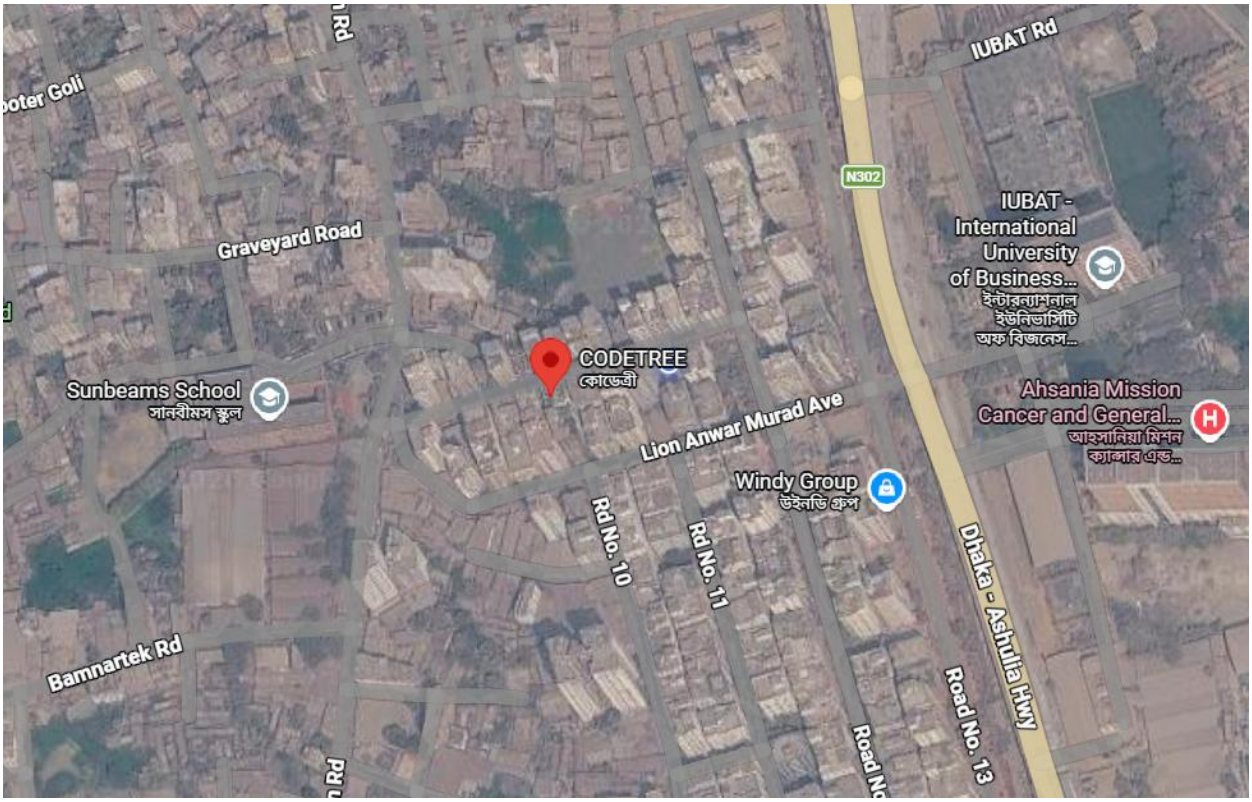


Figure 2.3: Organizational Location Map

Chapter 3.

Requirement Engineering

Requirement Engineering is the disciplined application of proven principles, methods, and tools (Society, 2014) to describe the behavior of the MessManager system and its associated constraints. This phase focused on understanding the actual needs of the stakeholders—specifically the Super Admin, Mess Manager, and members—to ensure the final product effectively solves their operational challenges regarding meal tracking and billing.

3.1 Requirement Elicitation & Analysis

This step involved gathering raw requirements from various sources to understand the functional depth of the system. The methods utilized included:

- **Stakeholder Analysis:** Identifying the key users—Super Admin, Manager, and Members—and understanding their specific privileges and workflows within the mess ecosystem.
- **Observation:** Observing manual processes currently used in messes, such as paper-based "Bazaar" lists and manual meal registers, to identify bottlenecks and calculation errors.
- **Conflict Resolution:** Potential conflicts, such as the need for detailed financial data vs. member privacy, were resolved by designing a robust role-based access control (RBAC) system.

3.2 Functional Requirements (FR)

These requirements define the specific functions and interactions within the system for different user roles.

Table 3.1 Functional Requirements

FR ID	Role	Description	Acceptance Criteria
FR-01	Public	Mess Creation Request	Manager can submit a request with mess name and email.

FR ID	Role	Description	Acceptance Criteria
FR-02	Super Admin	Request Approval	Super Admin can approve or reject pending mess requests.
FR-03	Manager	Member Management	Manager can add, update, or delete mess members.
FR-04	User	Meal Entry	Members can log Lunch, Dinner, or "Off" status for specific dates.
FR-05	Manager	Expense Tracking	Manager logs bazaar and other costs; titles labeled "Bazaar" impact meal rate.
FR-06	Manager	Fixed Cost Setup	Manager enters monthly rent, utilities, and WiFi costs to be split equally.
FR-07	User	Deposit Request	Members submit deposit requests which remain pending until approved.
FR-08	System	Billing Summary	System generates a monthly summary showing total bill, paid, and due.

3.3 Non-Functional Requirements

These requirements define the quality attributes and performance constraints of the system.

- **Security:** Passwords must be hashed using Bcrypt before storage, and critical routes must be protected by Laravel Sanctum middleware. (Otwell, 2023)
- **Data Integrity:** The database must ensure referential integrity; for example, an expense cannot exist without a valid Mess ID.

- **Performance:** The system should load dashboards within 2-3 seconds under normal network conditions.
- **Responsiveness:** The interface must be responsive, adapting its layout for desktops and mobile devices used by members on the go.
- **Availability:** The application must be available 24/7 to allow members to update meal statuses at their convenience.

3.4 Use Case Diagram

The Use Case Diagram illustrates the interactions between the primary actors and the core functionalities of the MessManager system.

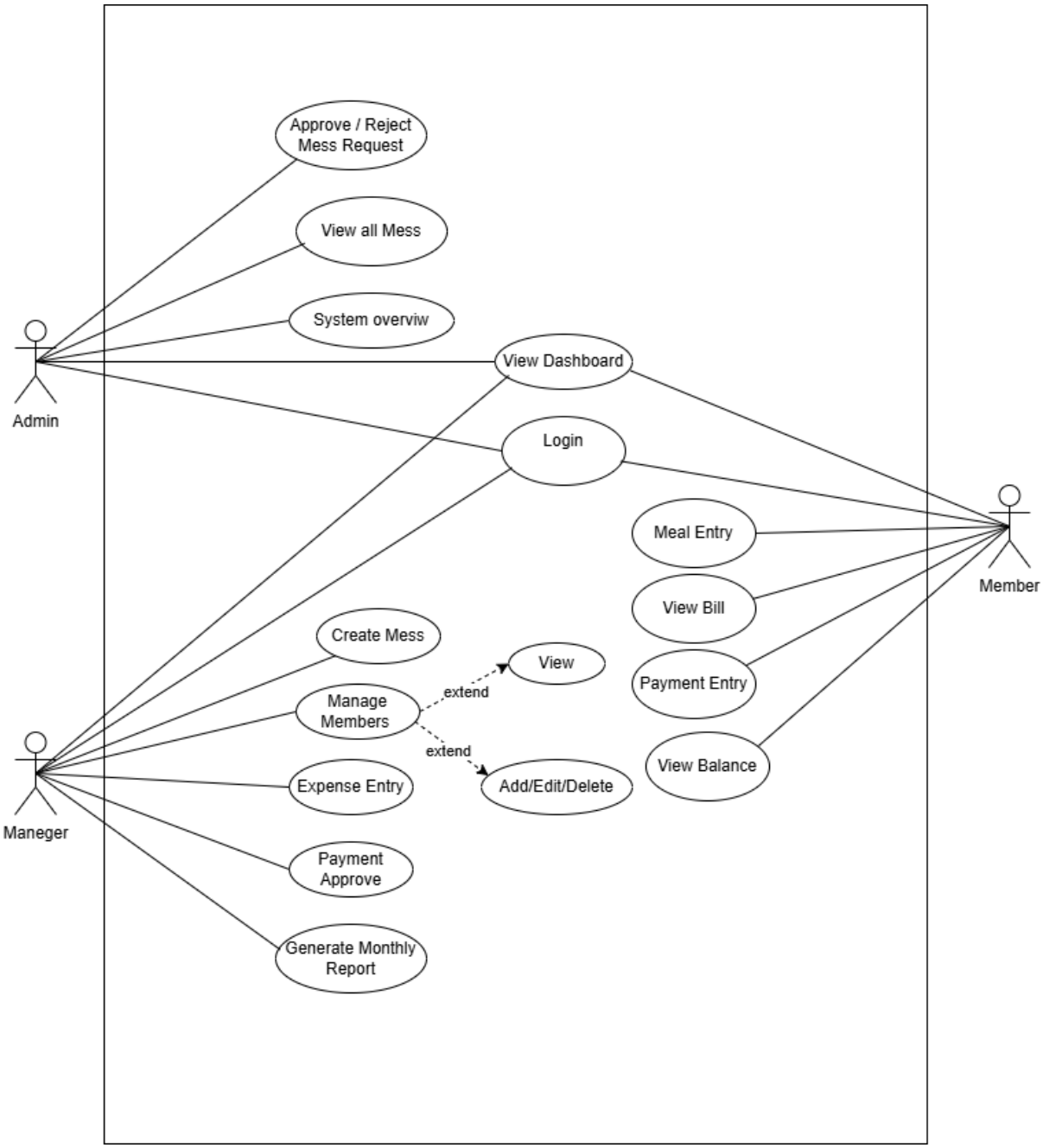


Figure 3.1: Use Case Diagram for MessManager

Chapter 4.

System Analysis

System analysis involves the creation of behavioural and structural models to represent the logic and data flow within the MessManager application. This chapter utilizes UML (Unified Modeling Language) diagrams and CRC cards to define how the system processes meal data, manages financial transactions, and organizes its internal object structure.

4.1 Activity Diagrams

Activity diagrams represent the operational workflows of the system, illustrating the transfer of control from the user to the system through sequential phases.

4.1.1: Admin Activity Diagram

This flow describes how a member updates their daily meal status:

1. **Member** selects a date from the calendar.
2. **Member** chooses status: Lunch, Dinner, or Off.
3. **System** validates if the date is within the current month.
4. **System** updates the meal quantity (Quantity is 0 if the status is Off).

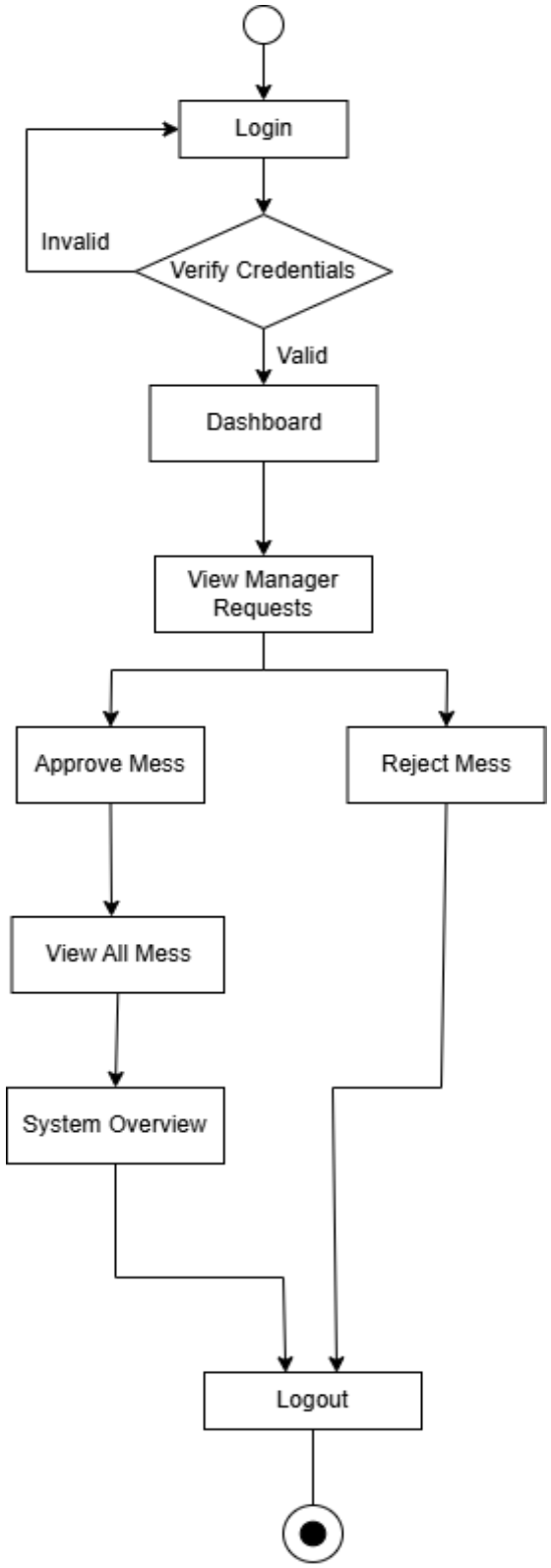


Figure 4.1.1: Activity Diagram for Admin

4.1.2: Manager Activity Diagram

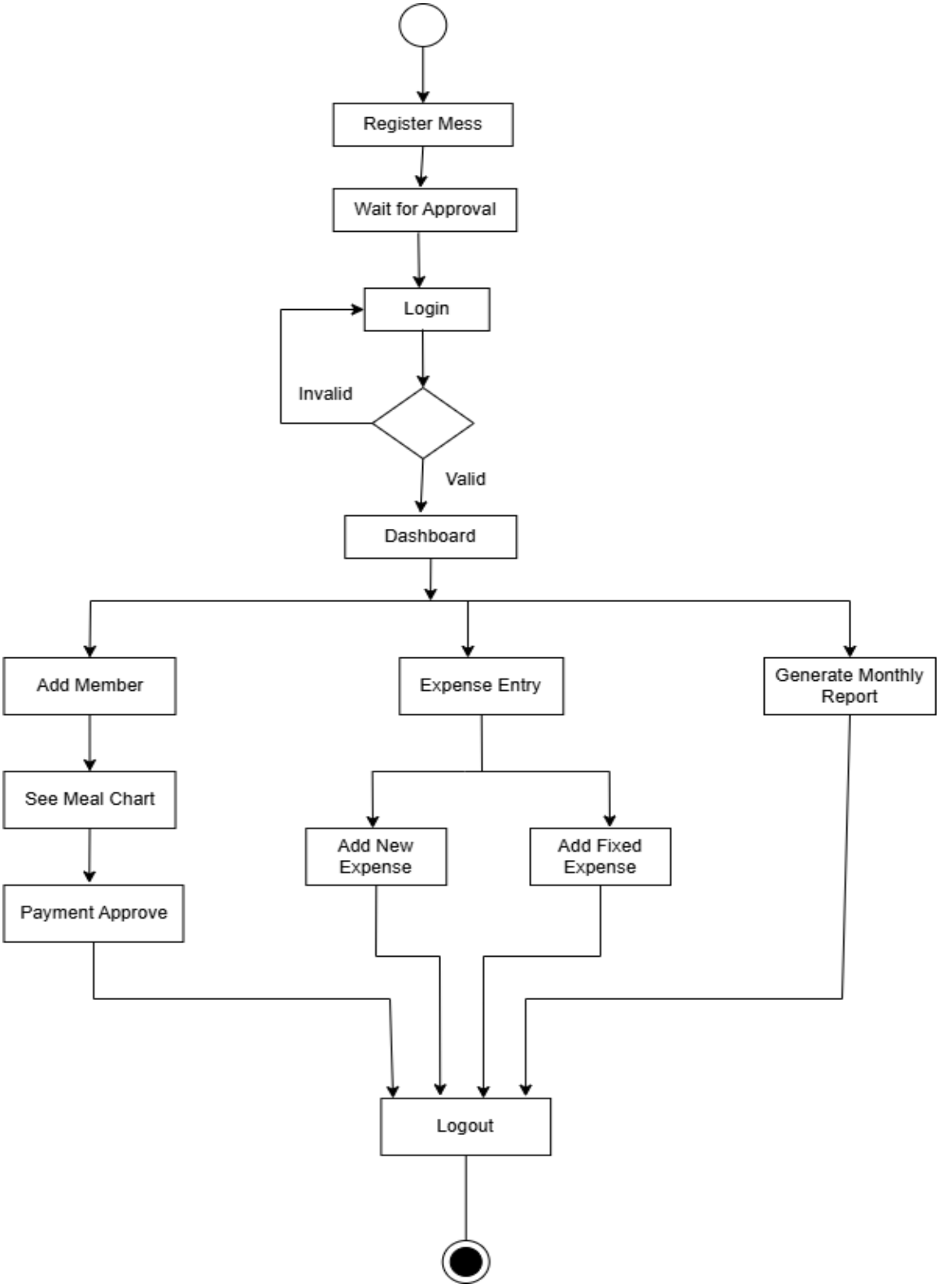


Figure 4.1.2: Activity Diagram for Manager

4.1.3: User Activity Diagram

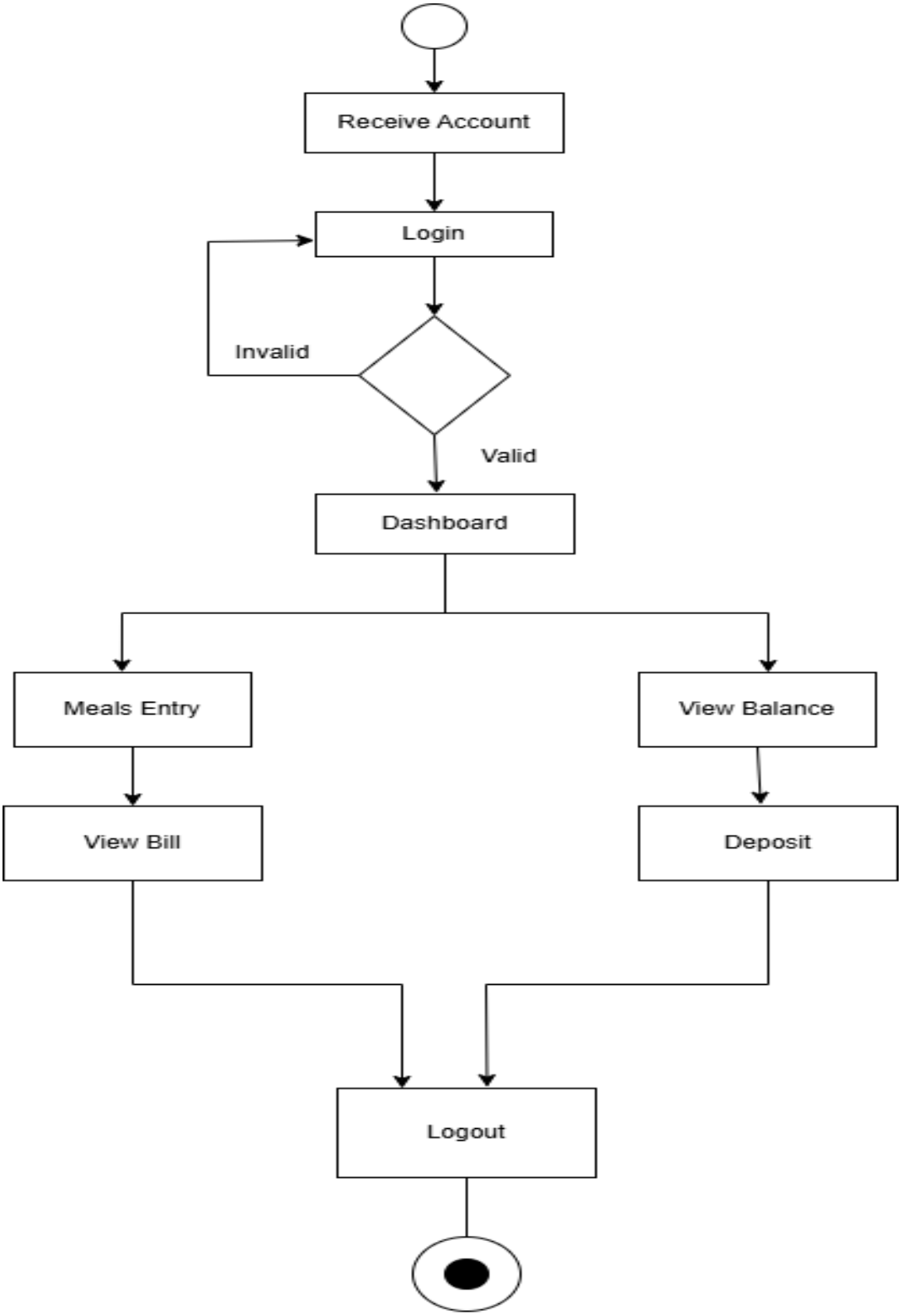


Figure 4.1.3: Activity Diagram for User

4.1.4: Meal Entry Activity Diagram

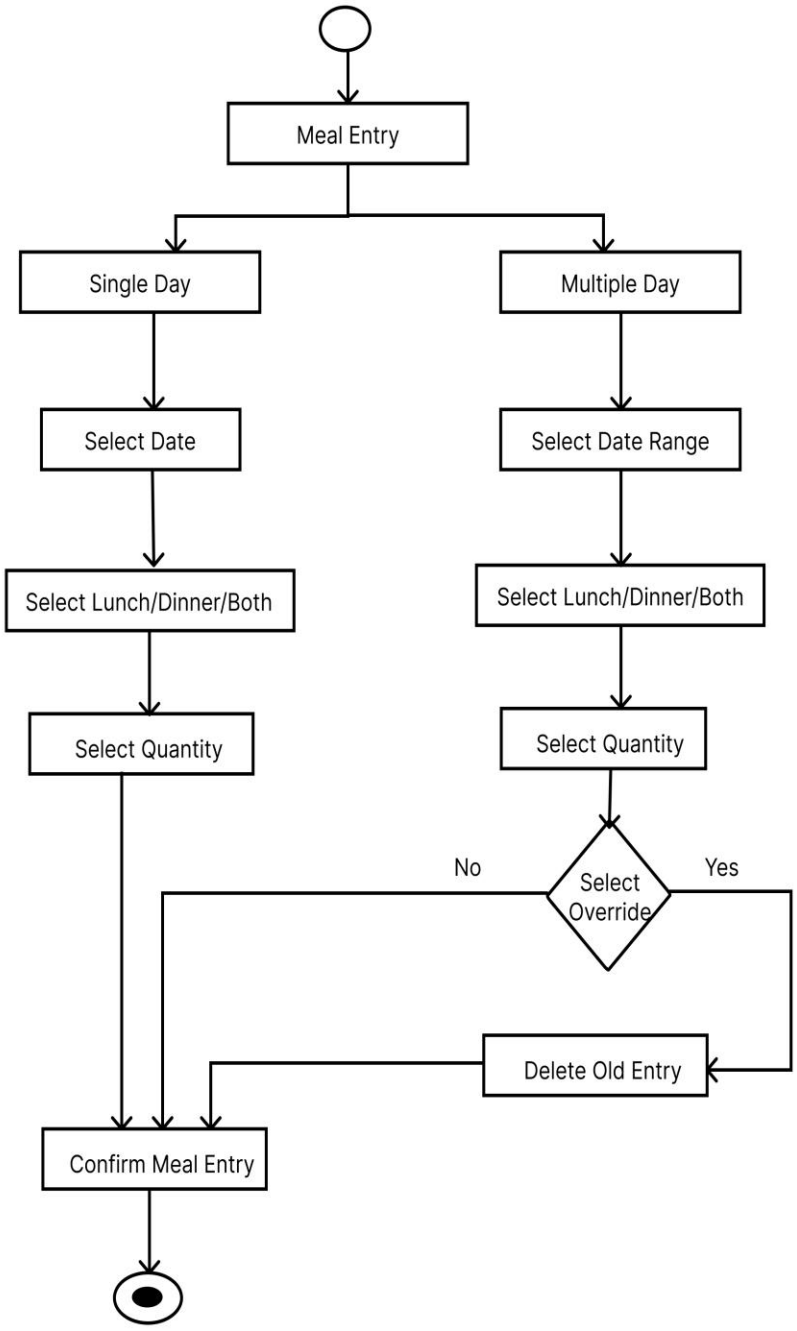


Figure 4.1.4: Activity Diagram for Meal Entry

4.1.5: Payment Processing Activity Diagram

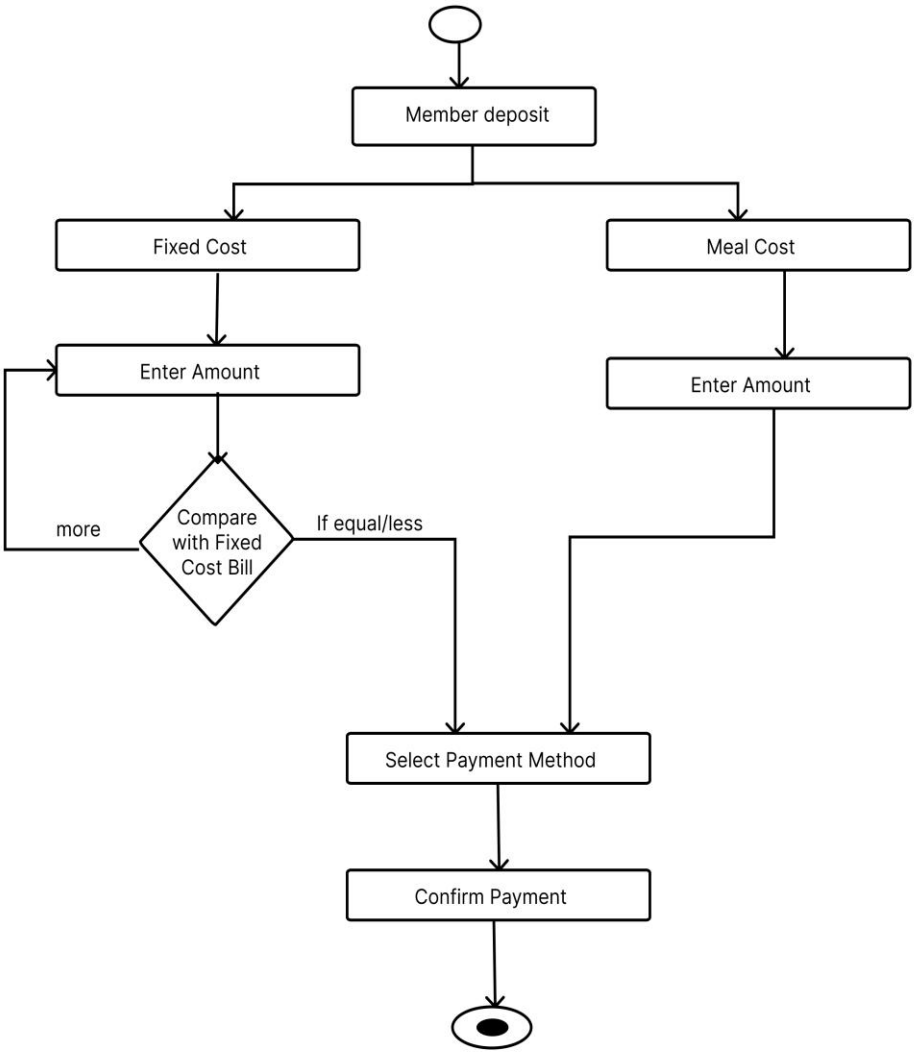


Figure 4.1.5: Activity Diagram for Payment Processing

4.2 Swimlane Diagram

Swimlane diagrams visualize role-based tasks, showing the coordinated responsibilities of different actors and the system.

4.2.1 Deposit Approval Flow

This diagram shows the interaction between the User (Member), the Manager, and the System for financial deposits:

- **User Lane:** Submits a deposit request with the amount.
- **System Lane:** Stores the request as Pending and notifies the Manager.
- **Manager Lane:** Reviews the request and clicks "Approve".
- **System Lane:** Creates a transaction record and updates the user's running balance.

4.2.2: Login & Authentication Swimlane Diagram

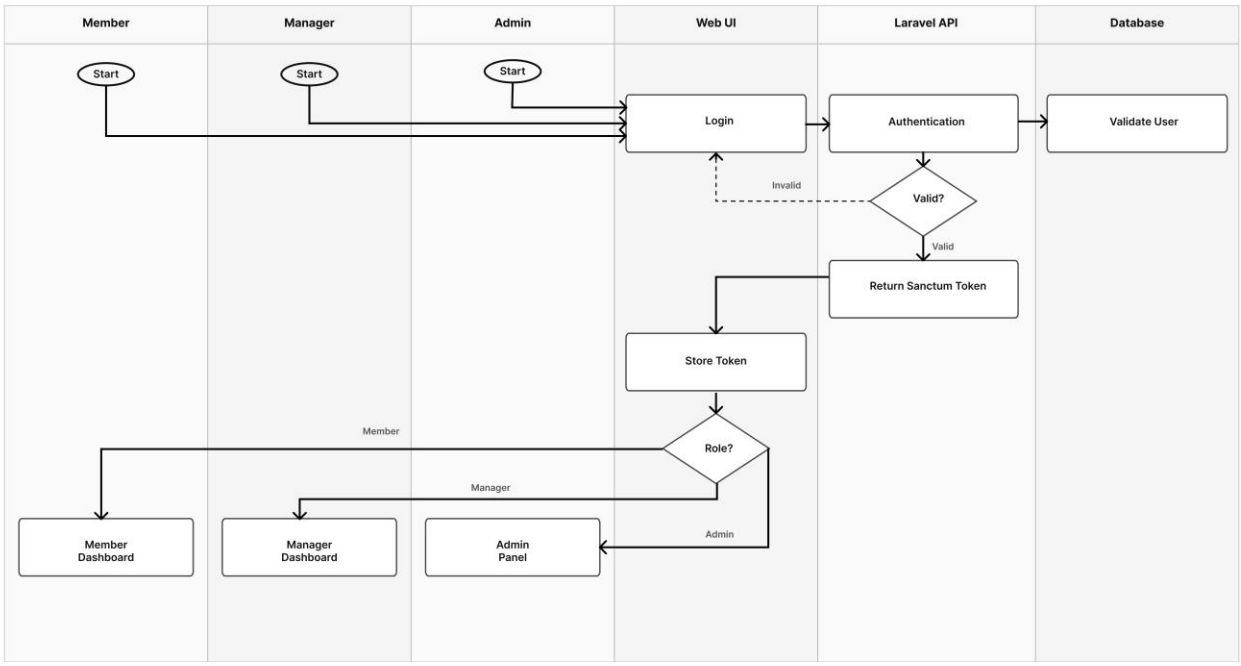


Figure 4.2.1: Swimlane Diagram for Login & Authentication

4.2.3: Mess Creation & Admin Approval Swimlane Diagram

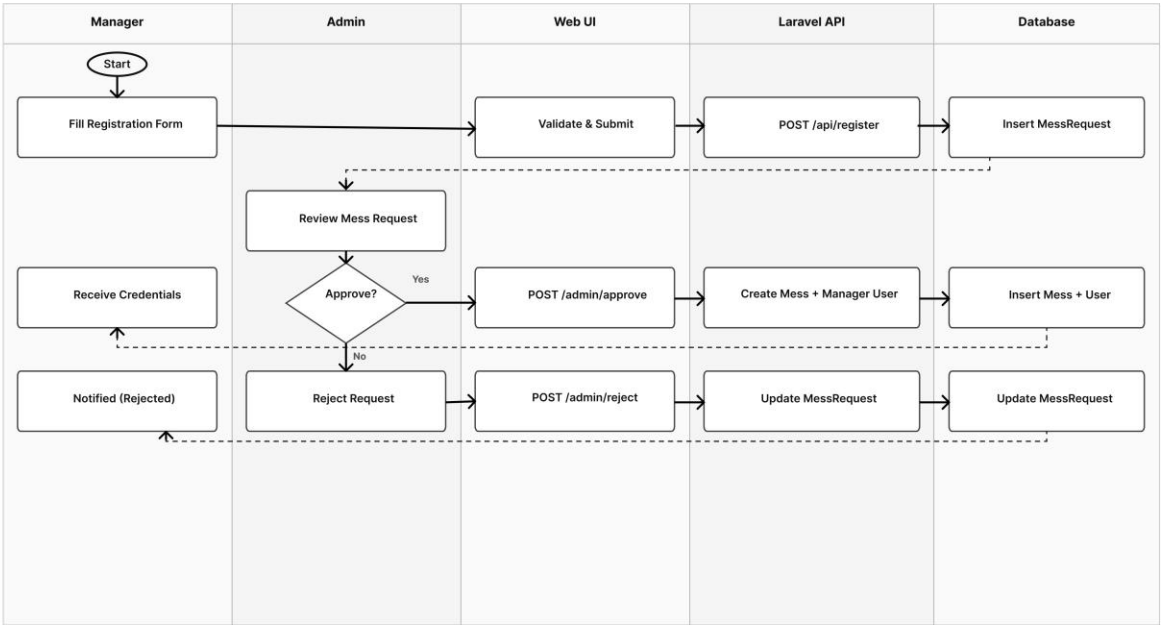


Figure 4.2.2: Swimlane Diagram for Mess Creation & Admin Approval

4.2.4: Member Onboarding Swimlane Diagram

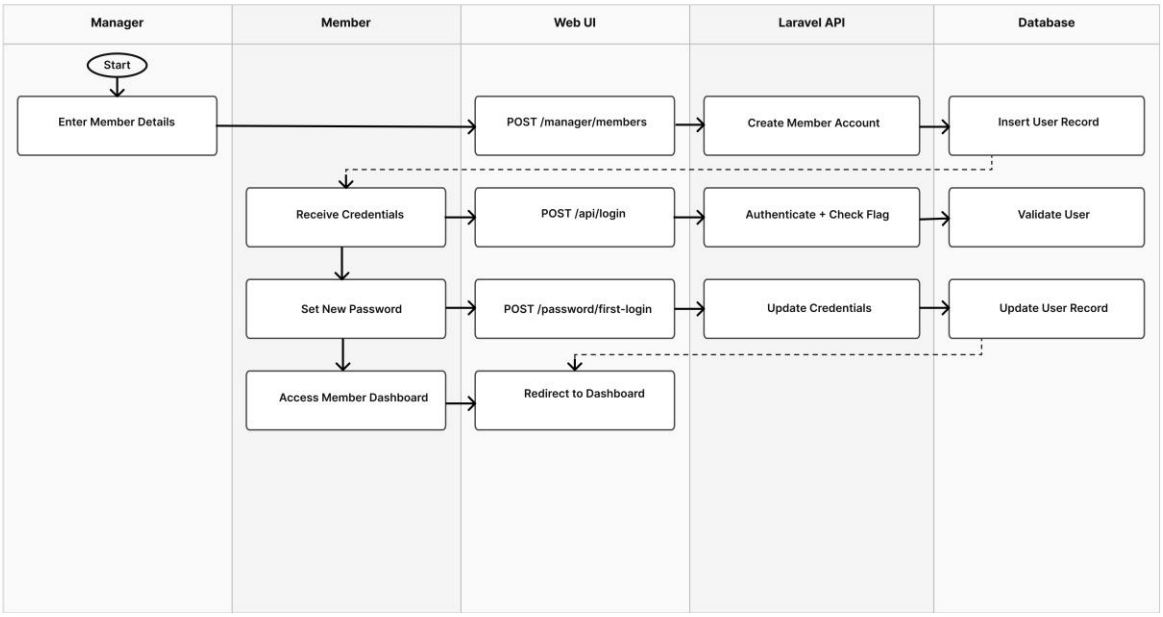


Figure 4.2.3: Swimlane Diagram for Member Onboarding

4.2.5: Meal Entry & Bill Tracking Swimlane Diagram

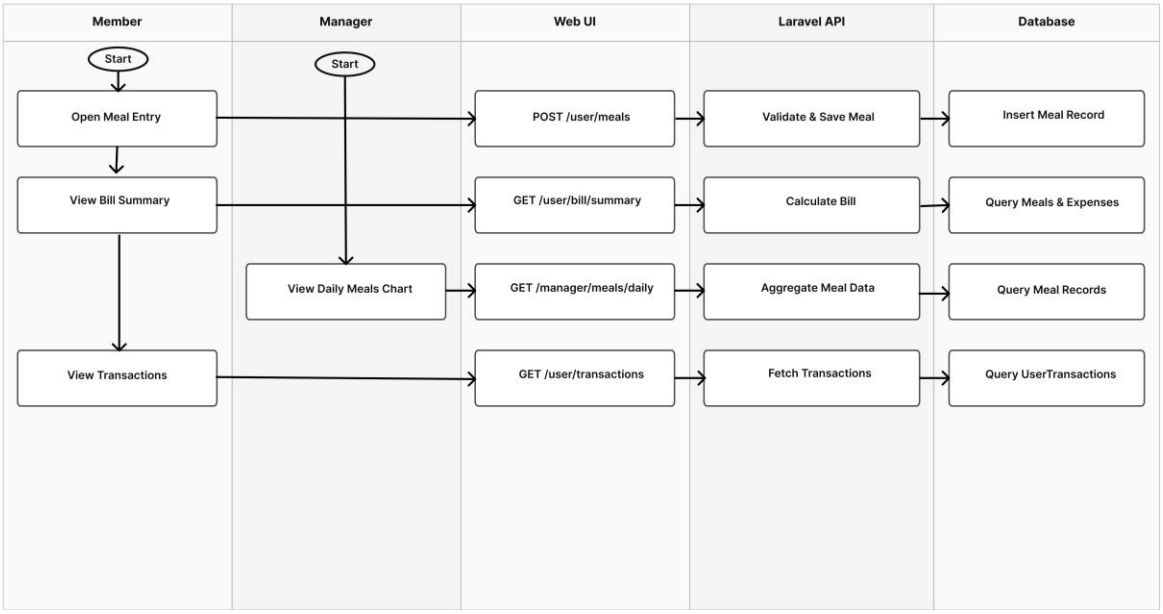


Figure 4.2.4: Swimlane Diagram for Meal Entry & Bill Tracking

4.2.6: Deposit Request & Approval Swimlane Diagram

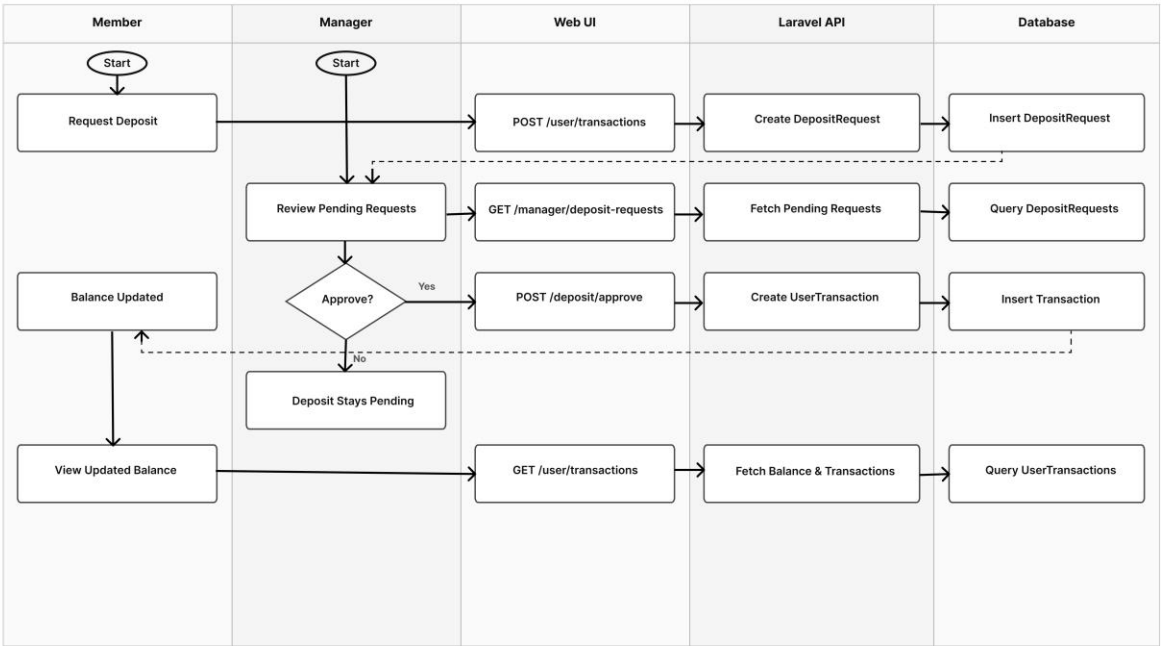


Figure 4.2.5: Swimlane Diagram for Deposit Request & Approval

4.2.7: Expense Management & Monthly Reports Swimlane Diagram

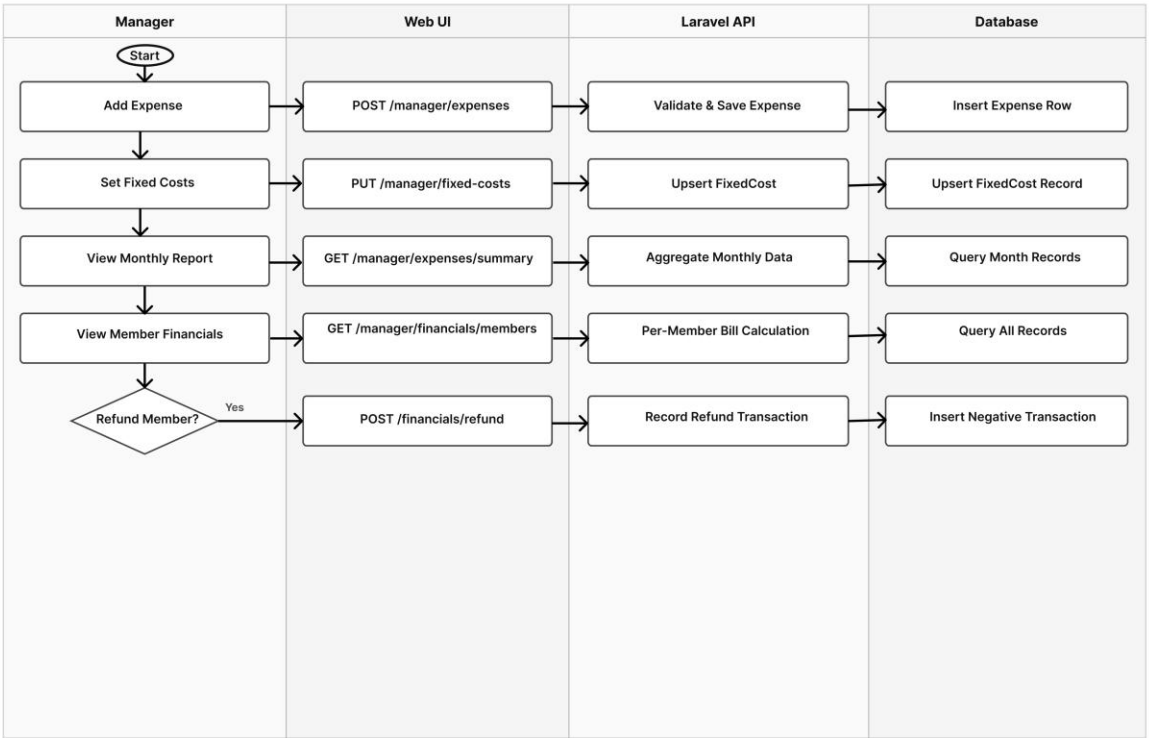


Figure 4.2.6: Swimlane Diagram for Expense Management & Monthly Reports

4.3 Class Diagram

The Class Diagram provides a structural view of the system, detailing the attributes and associations between core entities. (Corporation, 2023)

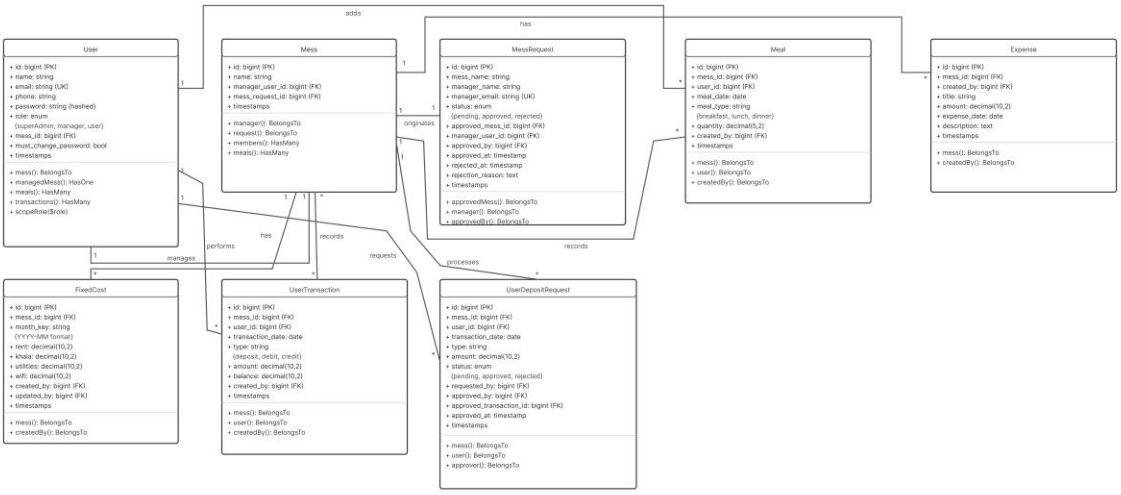


Figure 4.3: MessManager Class Diagram

4.4 Sequence Diagram

Sequence diagrams detail the logic behind complex system processes over a timeline.

4.4.1 Monthly Billing Calculation

This sequence tracks how the system computes a member's final bill:

1. **Manager** adds bazaar expenses for the month.
2. **Manager** adds fixed costs (Rent, WiFi, etc.).
3. **System** sums all bazaar costs and divides by total meals to get the Meal Rate.
4. **System** calculates the Fixed Share by dividing total fixed costs by member count.
5. **System** generates the Total Bill for each user: (\text{User Meals} \times \text{Rate}) + \text{Fixed Share}.

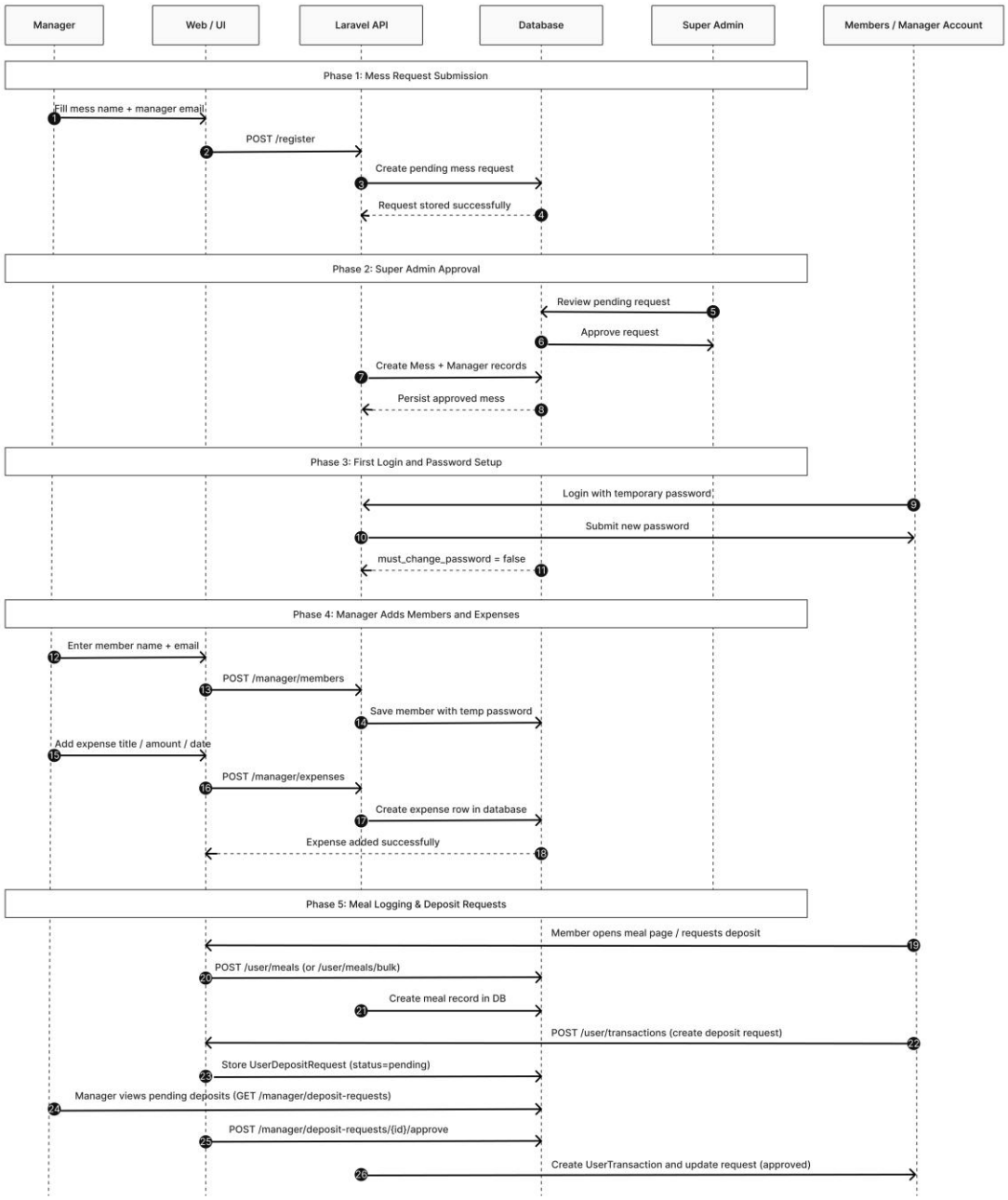


Figure 4.4: Billing Calculation Sequence Diagram

4.5 CRC Cards

CRC (Class-Responsibility-Collaborator) cards identify the specific duties of core classes and how they interact.

Table 4.1 CRC of User

User	
Responsibilities: # Authenticate with email/password # Set/change password # Maintain profile (name, email) # Track role (superAdmin, manager, user) # Manage must_change_password flag on first login # Provide access to mess data if manager or member # Store API tokens (Sanctum)	**Collaborators:** Mess Meal UserTransaction UserDepositRequest AuthService MemberManagementService

Table 4.2 CRC of Mess

Mess	
Responsibilities: # Store mess name and info # Track manager (User) # Reference mess request # Manage list of members # Track meals logged by members # Track expenses added # Track fixed costs monthly # Maintain transaction ledger	**Collaborators:** User MessRequest Meal Expense FixedCost UserTransaction

Table 4.3 CRC of MessRequest

MessRequest	
Responsibilities: # Store mess name requested # Store manager name/email # Track request status (pending/approved/rejected) # Reference approved Mess #Track approval decision (approved_by, approved_at) # Store rejection reason	**Collaborators:** Mess User MessRequestApprovalService

Table 4.4 CRC of Meal

Meal	
Responsibilities: # Store meal date # Store meal type (breakfast/lunch/dinner) # Store meal quantity # Track which mess # Track which user logged # Track who created record # Support meal cost calculation	**Collaborators:** Mess User BillingCalculator

Table 4.5 CRC of Expense

Expense	
Responsibilities: # Store expense title # Store expense amount # Store expense date # Store description # Track which mess # Track who created # Support categorization (bazaar, utilities, etc.)	**Collaborators:** Mess User ExpenseService BillingCalculator

Table 4.6 CRC of FixedCost

FixedCost	
Responsibilities: # Store month key (YYYY-MM) # Store rent amount # Store khala (helper) cost # Store utilities cost # Store wifi cost # Track which mess # Track who created/updated # Support monthly updates	**Collaborators:** Mess User BillingCalculator

Table 4.7 CRC of UserTransaction

UserTransaction	
Responsibilities: # Record transaction date # Store transaction type (deposit, debit, credit) # Store transaction amount # Maintain running balance # Track which mess # Track which user # Support balance calculation	**Collaborators:** Mess User TransactionManager

Table 4.8 CRC of UserDepositRequest

UserDepositRequest	
Responsibilities: # Store deposit amount # Track request status (pending/approved/rejected) # Track requested date # Track approval date # Track who approved # Store rejection reason # Track which user & mess	**Collaborators:** Mess User TransactionManager

Table 4.9 CRC of AuthService

AuthService	
Responsibilities: # Authenticate user (validate credentials) # Generate API tokens (Sanctum tokens) # Handle first-login password setup # Update user password # Validate user roles # Manage logout/token revocation	**Collaborators:** User Mess

Table 4.10 CRC of MemberManagementService

MemberManagementService	
Responsibilities: # Create member accounts # Validate member emails # Assign members to mess # Generate temp passwords # Update member details (name, email) # Delete member accounts # Validate manager permissions	**Collaborators:** User Mess AuthService

Table 4.11 CRC of BillingCalculator

BillingCalculator	
Responsibilities: # Calculate cost per meal (bazaar / total meals) # Sum total meals # Aggregate fixed costs # Calculate member share # Generate monthly bill for each member # Calculate breakdown (meals, fixed, other)	**Collaborators:** Meal Expense FixedCost Mess User

Table 4.12 CRC of BillingCalculator

TransactionManager	
Responsibilities: # Create transactions (deposit, debit, credit) # Maintain running balance # Process deposit requests # Validate deposit amounts # Generate balance statements # Get latest balance for a user/mess	**Collaborators:** UserTransaction UserDepositRequest User Mess BillingCalculator

Chapter 5.
Risk Management

Risk management is a critical process for identifying, analyzing, and addressing potential issues that could negatively impact the success of the MessManager project. By planning for risks early, the development team can minimize disruptions and ensure the system remains efficient and reliable.

5.1 Risk Identification

The following table outlines the potential risks categorized by their nature within the MessManager ecosystem.

Table 5.1 Risk Identification

Risk Type	Possible Risk
Technical	System crashes, Laravel/PHP framework issues, or MySQL database downtime.
Security	Data breaches, unauthorized access to member financial records, or token leakage.
Operational	Low user adoption, data entry errors by managers, or calculation discrepancies.
Financial	Failures in deposit tracking or future payment gateway integrations.
Performance	Slow response times during peak hours when all members log meals simultaneously.

5.2 Risk Analysis

Each identified risk is evaluated based on its probability of occurrence and its potential impact on the project.

Table 5.1 Risk Analysis

Possible Risk	Probability	Impact
System Downtime	30-40% (Moderate)	Serious

Possible Risk	Probability	Impact
Data Breaches	20-30% (Moderate)	Catastrophic
Unauthorized Edits	20-30% (Moderate)	Serious
Low User Adoption	30-40% (Moderate)	Tolerable

5.3 Risk Prioritization

Prioritization focuses on risks with high impact and moderate-to-high probability. For MessManager, Data Security and System Availability are the top priorities to ensure that sensitive meal and financial data remain protected and accessible 24/7.

5.4 RMMM Plan

The Risk Mitigation, Management, and Monitoring (RMMM) plan outlines the specific actions taken to handle the primary technical and security risks.

5.4.1 RMMM for Technical Risks (System Stability)

- **Risk Name:** System crashes or server downtime.
- **Mitigation:** Use of reliable cloud hosting and automated database backups to prevent data loss.
- **Monitoring:** Implement real-time monitoring tools to alert the developer of any service interruptions.
- **Management:** The developer will respond to alerts immediately and restore services using the latest backup if a crash occurs.

5.4.2 RMMM for Security Risks (Data Protection)

- **Risk Name:** Unauthorized access or data breaches.

- **Mitigation:** Use of Bcrypt for password hashing and Laravel Sanctum for secure, token-based authentication. (Group, 2021)
- **Monitoring:** Reviewing activity logs and login logs to detect suspicious patterns or unauthorized attempts to access manager routes.
- **Management:** If a breach is detected, the affected tokens will be revoked, and security patches will be applied to the middleware.

5.5 Monitoring & Review Plan

Risk management is an iterative process. The RMMM plan will be reviewed at the end of each development sprint to identify new risks—such as those arising from the future integration of OCR receipt scanning—and to ensure that current mitigation strategies remain effective.

Chapter 6.

Project Planning & Scheduling

Project planning and scheduling are essential to the management of software development operations. For MessManager, this phase involved establishing the project's objectives, resources, and timeline to ensure a functional product is delivered within the allotted three-month practicum period.

6.1 Function Point Estimation

Function Point Analysis (FPA) is a standardized method for measuring the functional size of software applications. For the MessManager system, we calculate the function points to estimate the project size and effort.

6.1.1 Functionality, Input, Output

The table below shows the core functionality, inputs, and outputs of the MessManager system.

Table 6.1 Functionality, Input, Output

Functionality	Input	Output
Mess Registration	messName, managerEmail, managerName	Pending mess request created
User Authentication	email, password	Login successful, dashboard redirect
Member Management	name, email, temporaryPassword	User account created
Meal Entry	date, mealType (Lunch/Dinner/Off)	Daily meal logged
Expense Entry	title, amount, date	Expense added (Bazaar/Other)
Fixed Cost Entry	rent, utilities, wifi, monthKey	Monthly fixed costs stored

Functionality	Input	Output
Deposit Request	amount, paymentMethod	Pending deposit submitted
Bill Summary	monthKey	Total bill, paid, and due summary

6.1.2 Identify Complexity

Complexity is determined by identifying Data Element Types (DET) and Record Element Types (RET) for Data Functions, and File Types Referenced (FTR) for Transaction Functions.

Table 6.2: Identify Complexity of Data Function

Function Name	Types	DET	RET	Complexity
Users	ILF	12	1	Low
Meals	ILF	12	1	Low
Expenses	ILF	10	1	Low
Fixed Costs	ILF	9	1	Low
Transactions	ILF	10	1	Low
Mess Requests	ILF	8	1	Low

Table 6.3: Identify Complexity of Transaction Function

Function Name	Type	DET	FTR	Complexity
Mess Registration	EI	5	1	Low
User Authentication	EI	2	2	Low
Meal Entry	EI	8	3	Average
Expense Entry	EI	6	2	Average
Fixed Cost Entry	EI	5	2	Average
Deposit Request	EI	5	2	Average
Bill Summary	EQ	4	2	Low

6.1.3 Unadjusted Function Point Contribution

The Unadjusted Function Points (UFP) are calculated by applying standard weights to the identified functions.

Table 6.4: UFP of Transaction Function

Function Name	Type	DET	FTR	Complexity	UFP
Mess Registration	EI	5	1	Low	3
User Authentication	EI	2	2	Low	3

Function Name	Type	DET	FTR	Complexity	UFP
Meal Entry	EI	8	3	Average	4
Expense Entry	EI	6	2	Average	4
Fixed Cost Entry	EI	5	2	Average	4
Deposit Request	EI	5	2	Average	4
Bill Summary	EQ	4	2	Low	3
Total UFP (Transaction)					25

Table 6.5: UFP of Data Function

Function Name	Type	DET	RET	Complexity	UFP
Users	ILF	12	1	Low	7
Meals	ILF	12	1	Low	7
Expenses	ILF	10	1	Low	7
Fixed Costs	ILF	9	1	Low	7
Transactions	ILF	10	1	Low	7
Total UFP (Data)					35

6.1.4 Calculation of Total Degree of Influence (TDI)

The degree of influence is calculated based on 14 general system characteristics.

Table 6.6: UFP of Data Function

SI	Characteristic	DI	SI	Characteristic	DI
01	Data Communication	5	08	Online Update	5
02	Distributed Data Processing	3	09	Complex Processing	4
03	Performance	4	10	Reusability	3
04	Heavily Used Configuration	4	11	Installation Ease	3
05	Transaction Rate	4	12	Operational Ease	4
06	Online Data Entry	5	13	Multiple Sites	2
07	End-User Efficiency	5	14	Facilitate Change	4
Total TDI					55

6.1.5 Final Calculation

The Value Adjustment Factor (VAF) and Adjusted Function Point (AFP) are calculated as follows:

- VAF = 0.65 + (0.01 \times TDI)
- VAF = 0.65 + (0.01 \times 55) = 1.10
- Total UFP = UFP(Transaction) + UFP(Data) = 25 + 35 = 60
- AFP = UFP \times VAF = 60 \times 1.10 = 66

The final functional size of the MessManager system is 66 Adjusted Function Points.

6.2 Project Scheduling

The MessManager development was planned over a 12-week timeline (3 months) following the Agile methodology. The project is divided into six distinct phases to ensure iterative testing and quality assurance.

6.2.1 Gantt Chart Breakdown

The following 12-week schedule outlines the progression from requirements gathering to final deployment.

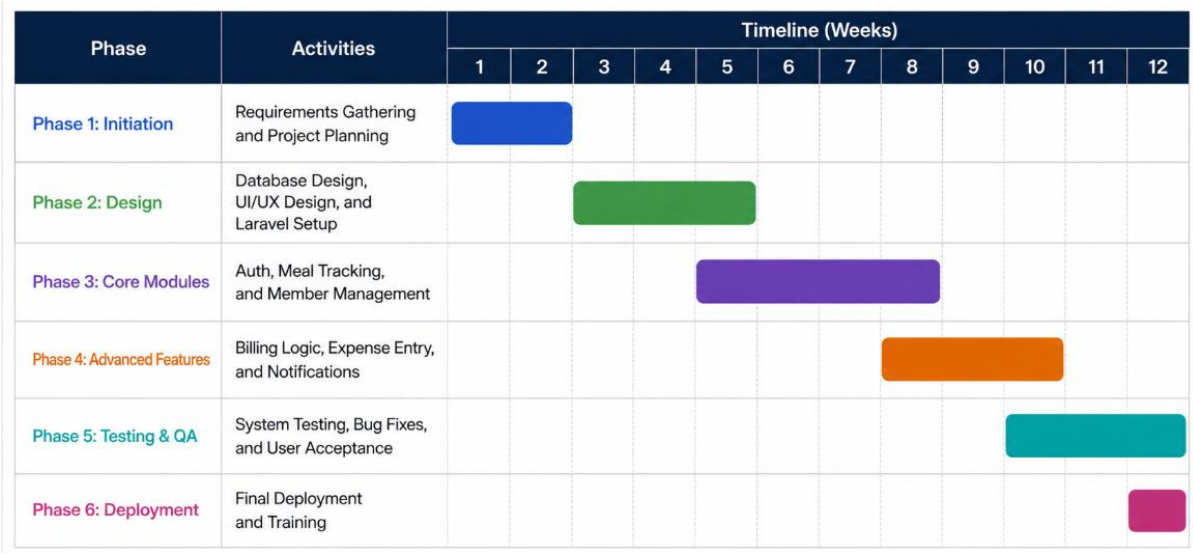


Figure 6.1: MessManager 12-Week Gantt Chart

6.3 Resource & Cost Estimate

The project utilized open-source technologies (Laravel, React, MySQL) which significantly reduced the financial burden. The primary resources included the development environment (VS Code, PHP 8.1+, Node.js) and the staging server for initial testing. This approach ensured that the project remained economically sustainable while meeting all functional objectives

Chapter 7.

System Design & Implementation

System design and implementation translate the logical requirements into a physical, functioning application. This chapter details the technical architecture, data structures, and interface components of the MessManager system, highlighting how the Laravel backend and React frontend interact to provide a seamless user experience.

7.1 System Architecture

The application follows a decoupled, layered architecture to ensure scalability and maintainability. It is built as a Single Page Application (SPA) that communicates with a RESTful API.

- **Frontend Layer (React 19 + Vite):** Handles the user interface, routing, and client-side state management using Redux Toolkit.
- **API Layer (Laravel 10):** Provides secure endpoints, business logic processing, and authentication via Laravel Sanctum.
- **Database Layer (MySQL):** Stores persistent data such as user profiles, meal records, and financial transactions.

7.2 Data Flow Diagrams (DFD)

Data Flow Diagrams illustrate how information moves through the system, from external inputs to database storage.

- **Level 0 (Context Level):** Shows the high-level interaction between actors (Manager, Member, Super Admin) and the MessManager system.

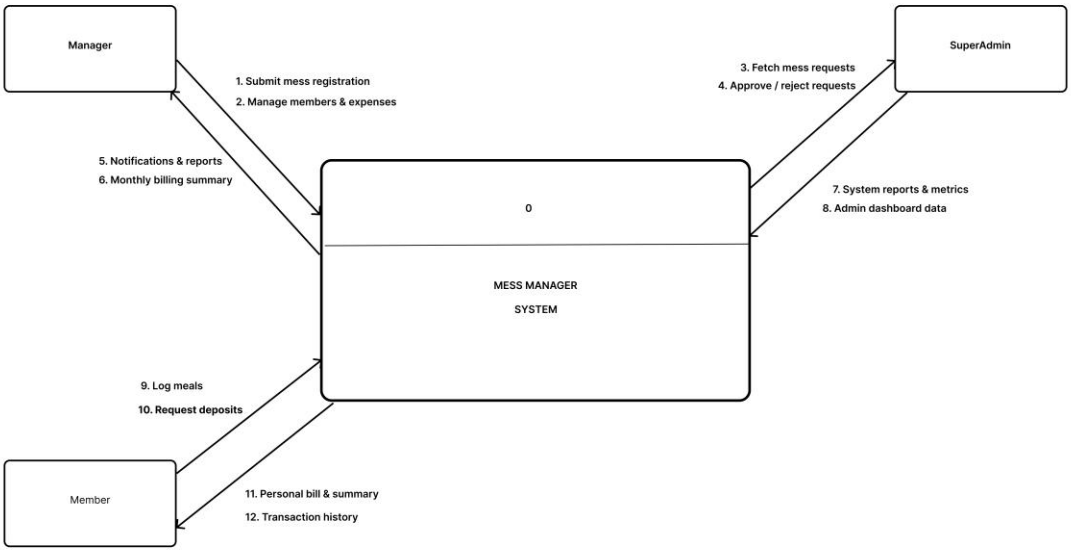


Figure 7.1 DFD Level 0

- **Level 1 (Process Overview):** Breaks down the system into core modules like Auth, Meal Management, and Billing.

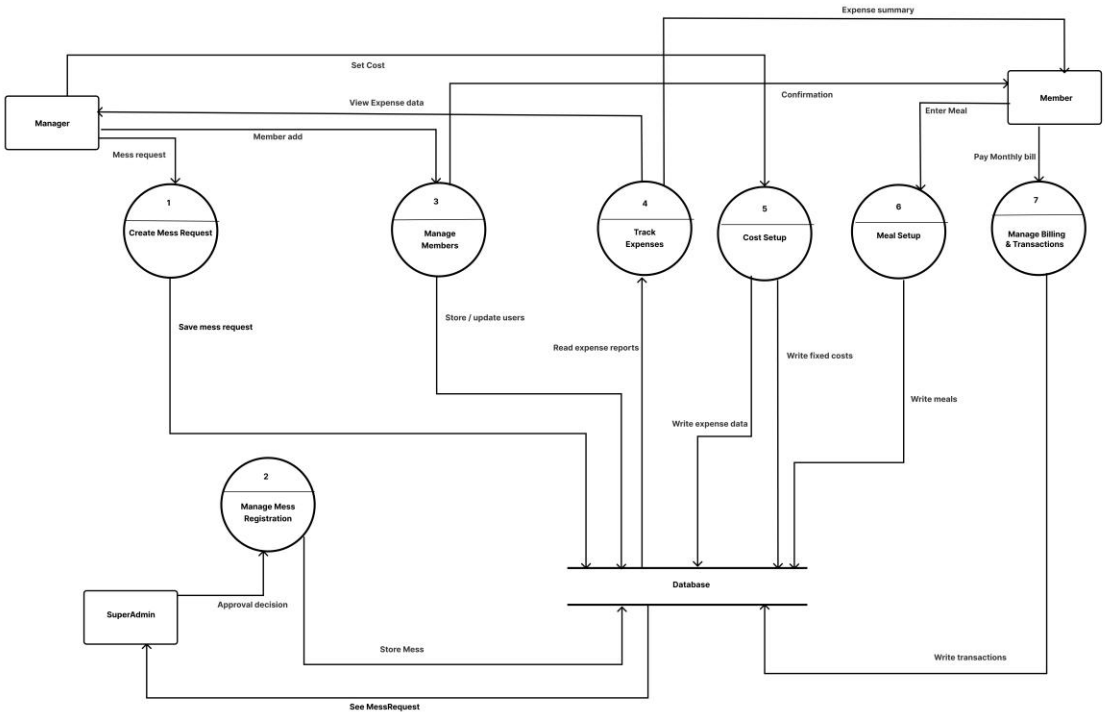


Figure 7.2 DFD Level 1

- **Level 2 (Mess Creation & Approval):**

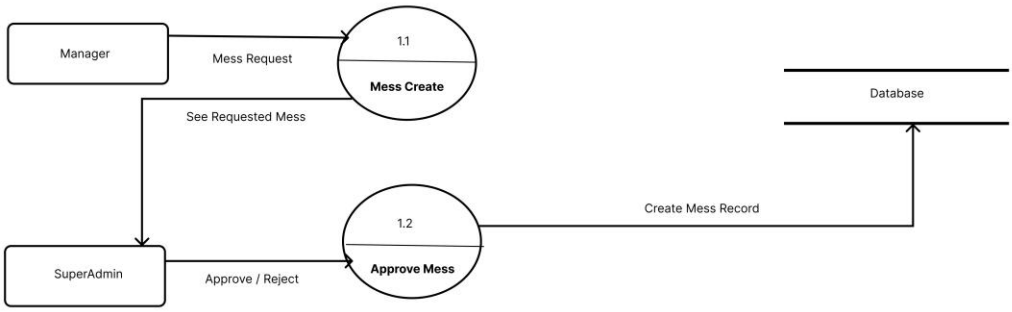


Figure 7.3 DFD Level 2 (1)

- **Level 2 (Mess Registration Management):**

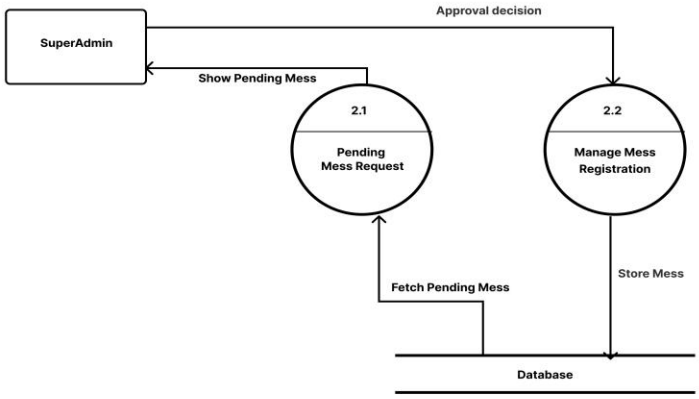


Figure 7.4 DFD Level 2 (2)

- **Level 2 (Member Management):**

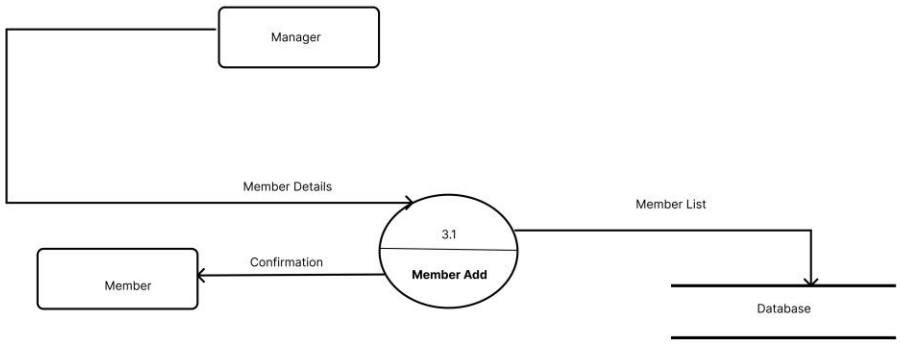


Figure 7.5 DFD Level 2 (3)

- **Level 2 (Expense Management):**

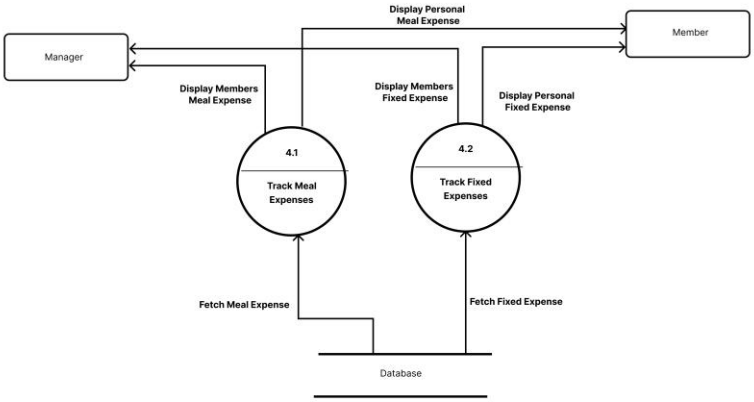


Figure 7.6 DFD Level 2 (4)

- **Level 2 (Cost Setup & Meal Cost Calculation):**

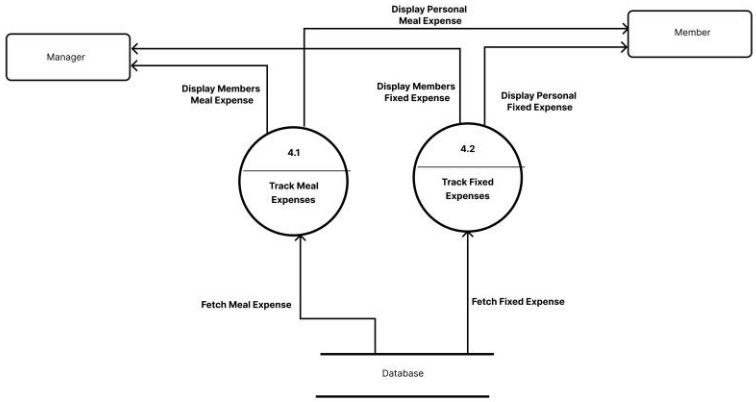


Figure 7.7 DFD Level 2 (5)

- **Level 2 (Meal Management):**

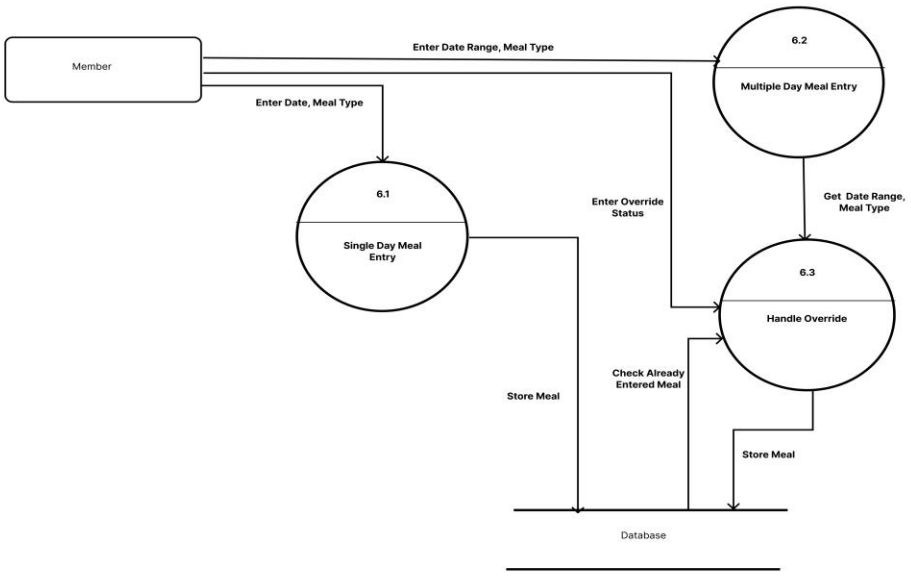


Figure 7.8 DFD Level 2 (6)

- **Level 2 (Billing & Transaction Management):**

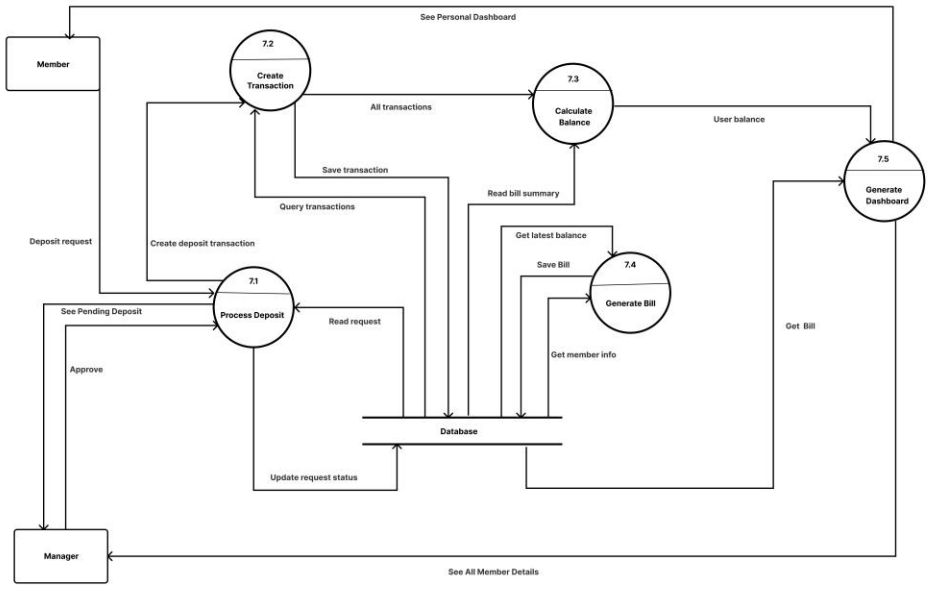


Figure 7.9 DFD Level 2 (7)

7.3 Entity Relationship Diagram (ERD)

The ERD defines the database schema and the relationships between tables. Key relationships include: (Corporation, 2023)

- **One-to-Many:** A Mess has many Users, Meals, and Expenses.
- **One-to-One:** Each User is associated with one primary Mess group and one specific Role.

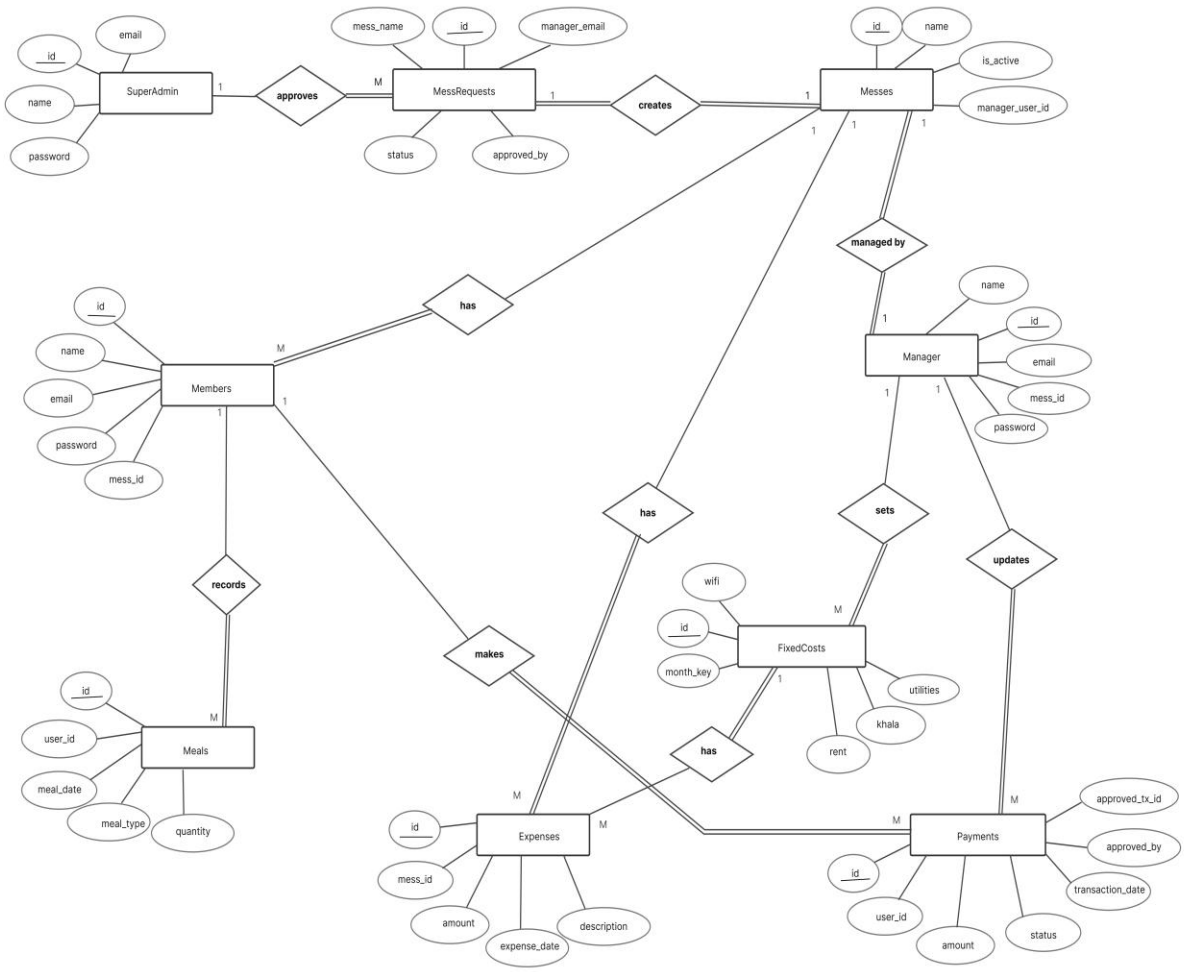


Figure 7.10 Entity Relationship Diagram

7.3.1 Database Field Design

Filters

Containing the word:

Table	Action	Rows	Type	Collation	Size	Overhead
expenses	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
failed_jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
fixed_costs	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
meals	Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
messes	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
mess_requests	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
migrations	Browse Structure Search Insert Empty Drop	13	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
password_reset_tokens	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
personal_access_tokens	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
users	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
user_deposit_requests	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_unicode_ci	96.0 KiB	-
user_transactions	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
12 tables	**Sum**	**46**	**InnoDB**	**utf8mb4_general_ci**	**608.0 KiB**	**0 B**

Figure 7.11 Database field design

Showing rows 0 - 5 (6 total, Query took 0.0003 seconds.)

```
SELECT * FROM `users`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	id	name	email	phone	email_verified_at	password	role	me
Edit Copy Delete	1	Super Admin	superadmin@mess.com	NULL	NULL	$2y$12$eAzAUSgcvCKIeLIDj6xg5ub56c.LWrq0GXMLFEJb8Zo...	superAdmin	
Edit Copy Delete	5	Hasan Imam	hasan@gmail.com	NULL	NULL	$2y$12$x9K1yT6EjX1HwqXSc3tcZuakC7cTCIRZt0qmicuYhUO...	manager	
Edit Copy Delete	6	Laden	laden@gmail.com	NULL	NULL	$2y$12$3u//ElE4nOtB6OULA/zJzuWMm80TXo8f2cgtJGglQj2...	user	
Edit Copy Delete	7	Asik	asik@gmail.com	NULL	NULL	$2y$12$0UCxS9oiLDHZddJ3QLPZdOTv7ugKbs5v9.6Nc72PPGi...	manager	
Edit Copy Delete	8	Imam Hasan	imam@gmail.com	NULL	NULL	$2y$12$Dg2uI5eBXqPXo7ZTkKQrg.C3xz/GsbAoMgszgWseJ8Z...	user	
Edit Copy Delete	9	Nafiz	nafiz@gmail.com	NULL	NULL	$2y$12$bK8AnmG521tdDXcWyqf4r.mZShTyUu4bqED3sz2blZL...	user	

Check all With selected: Edit Copy Delete Export

Figure 7.12 User Table

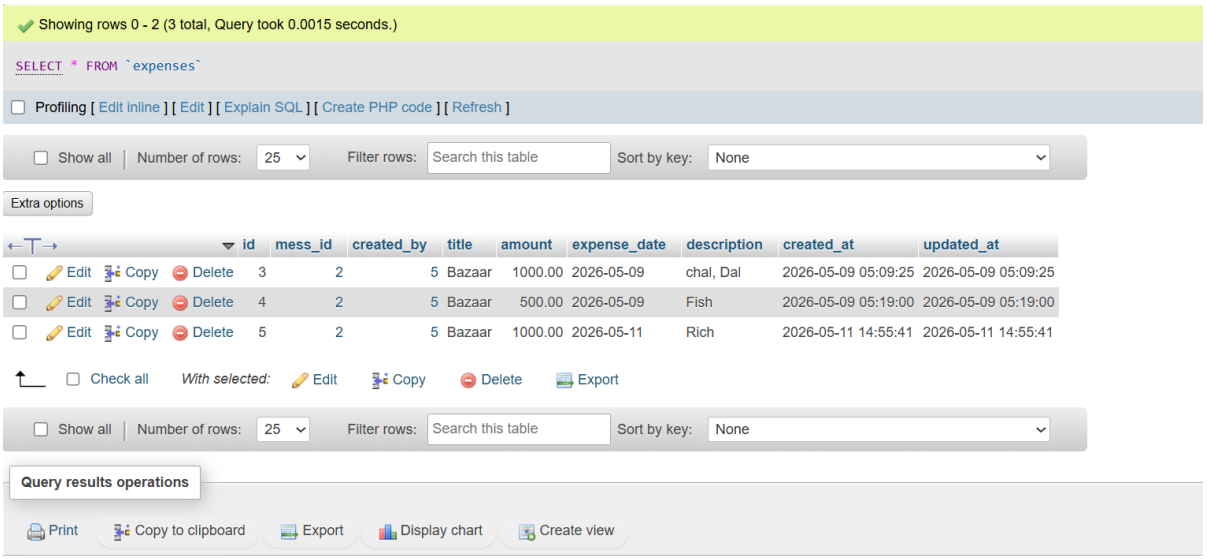


Figure 7.13 Expense Table

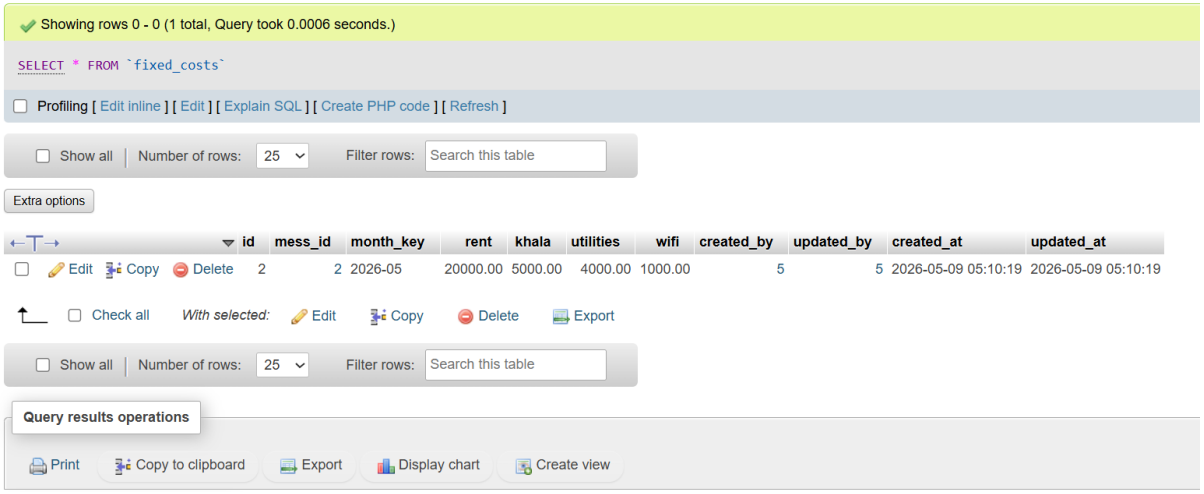


Figure 7.14 Fixed Costs Table

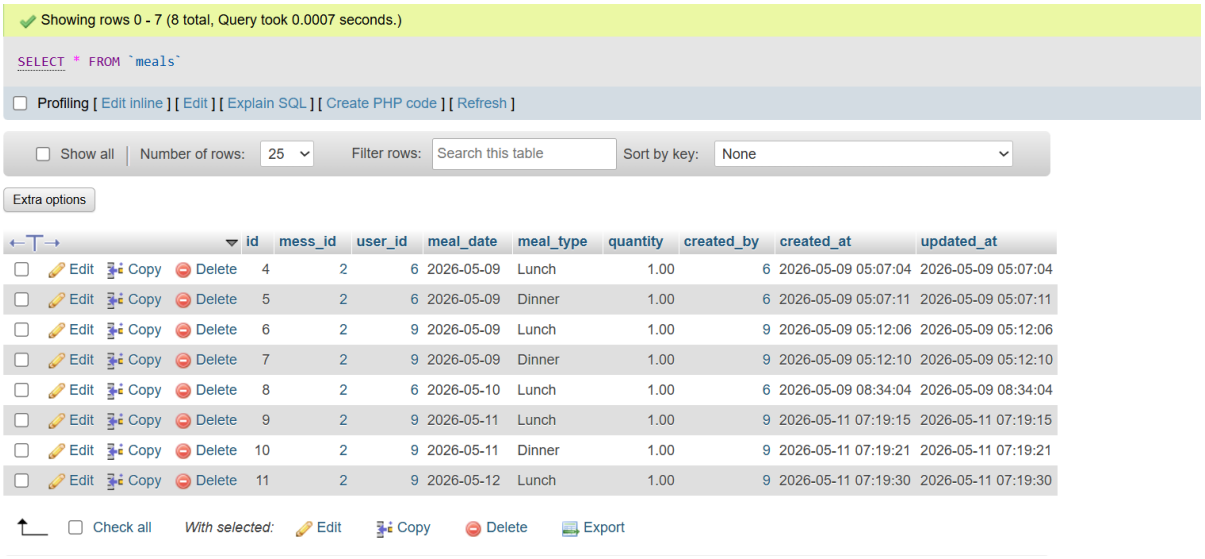


Figure 7.15 Meals Table

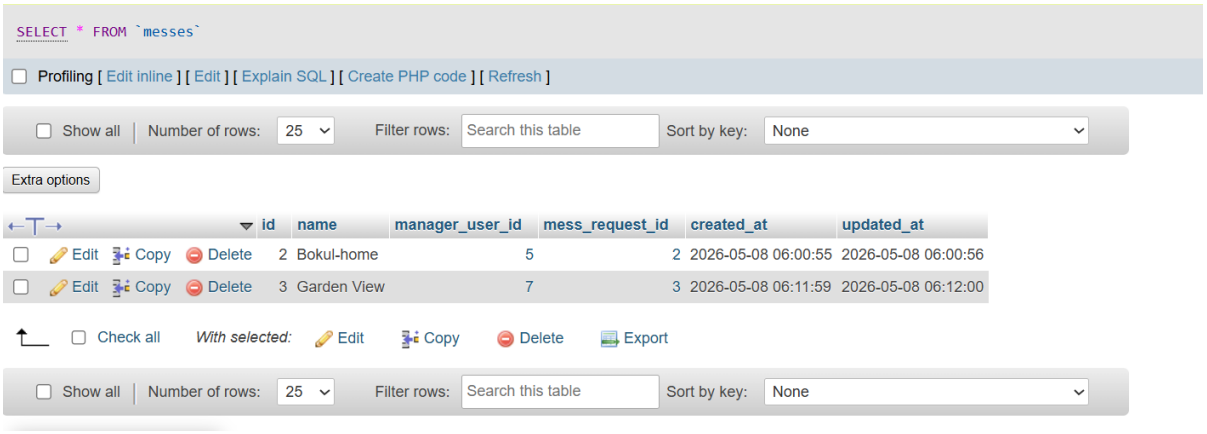


Figure 7.16 Messes Table

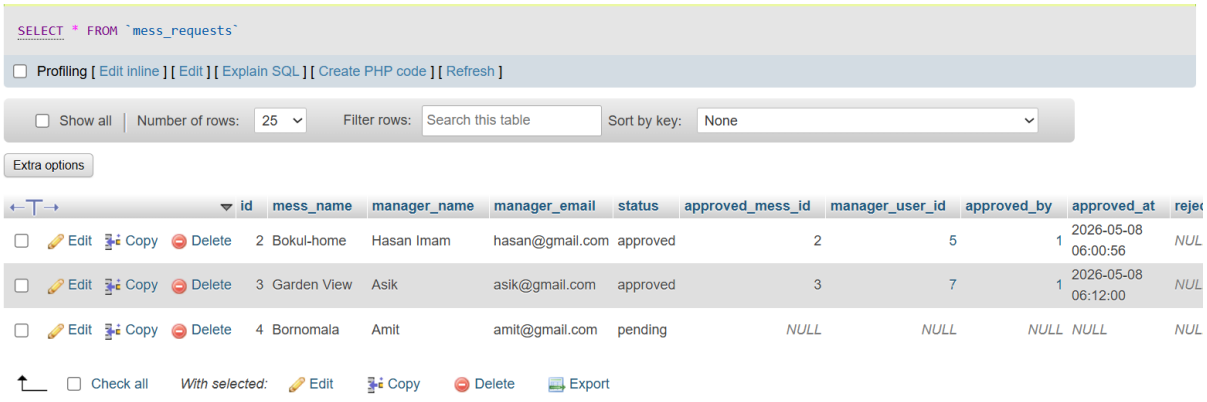
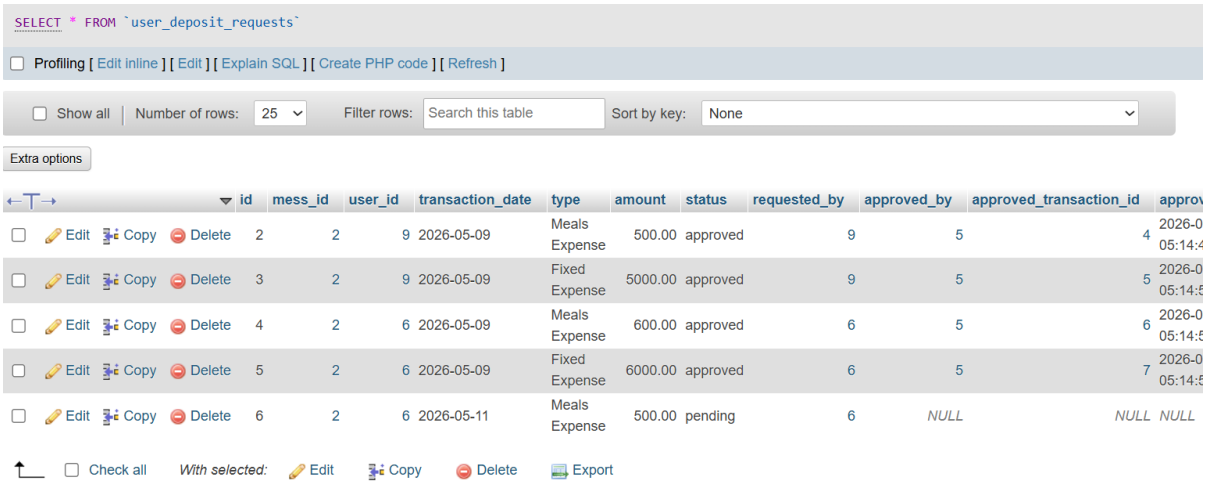


Figure 7.17 Mess Request Table



```
SELECT * FROM `user_deposit_requests`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

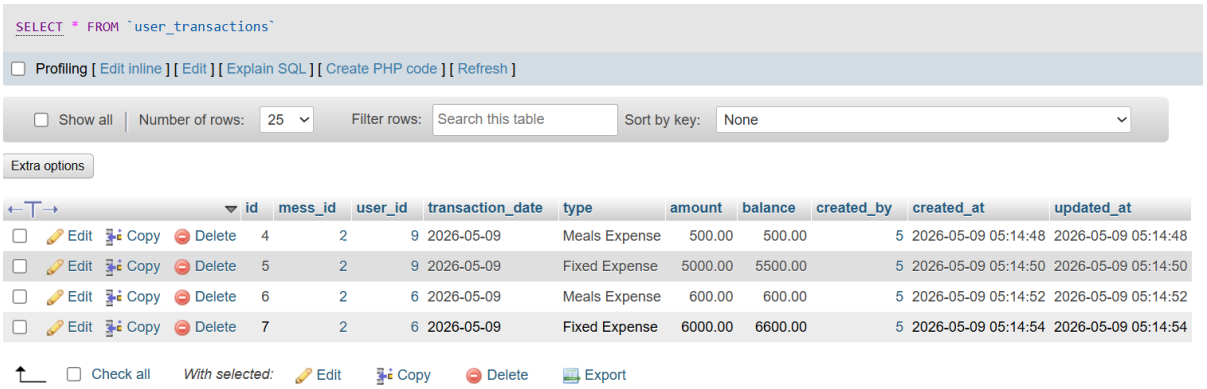
Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	id	mess_id	user_id	transaction_date	type	amount	status	requested_by	approved_by	approved_transaction_id	approv
Edit Copy Delete	2	2	9	2026-05-09	Meals Expense	500.00	approved	9	5	4	2026-0 05:14:4
Edit Copy Delete	3	2	9	2026-05-09	Fixed Expense	5000.00	approved	9	5	5	2026-0 05:14:5
Edit Copy Delete	4	2	6	2026-05-09	Meals Expense	600.00	approved	6	5	6	2026-0 05:14:5
Edit Copy Delete	5	2	6	2026-05-09	Fixed Expense	6000.00	approved	6	5	7	2026-0 05:14:5
Edit Copy Delete	6	2	6	2026-05-11	Meals Expense	500.00	pending	6	NULL	NULL	NULL

Check all With selected: Edit Copy Delete Export

Figure 7.18 User_Deposite_Requests



```
SELECT * FROM `user_transactions`
```

Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

Show all | Number of rows: 25 Filter rows: Search this table Sort by key: None

Extra options

	id	mess_id	user_id	transaction_date	type	amount	balance	created_by	created_at	updated_at
Edit Copy Delete	4	2	9	2026-05-09	Meals Expense	500.00	500.00	5	2026-05-09 05:14:48	2026-05-09 05:14:48
Edit Copy Delete	5	2	9	2026-05-09	Fixed Expense	5000.00	5500.00	5	2026-05-09 05:14:50	2026-05-09 05:14:50
Edit Copy Delete	6	2	6	2026-05-09	Meals Expense	600.00	600.00	5	2026-05-09 05:14:52	2026-05-09 05:14:52
Edit Copy Delete	7	2	6	2026-05-09	Fixed Expense	6000.00	6600.00	5	2026-05-09 05:14:54	2026-05-09 05:14:54

Check all With selected: Edit Copy Delete Export

Figure 7.19 User Transaction Table

7.4 Interface Design

The user interface is designed to be intuitive and role-specific, ensuring that members only see features relevant to them.

- **Manager Dashboard:** Provides a bird's-eye view of mess statistics, member lists, and pending deposit approvals.
- **Member Panel:** Focuses on meal logging, viewing current personal balances, and requesting deposits.
- **Billing Summary:** A tabular view showing the monthly breakdown of bazaar costs, fixed costs, and the calculated meal rate.

7.4.1 Home Page

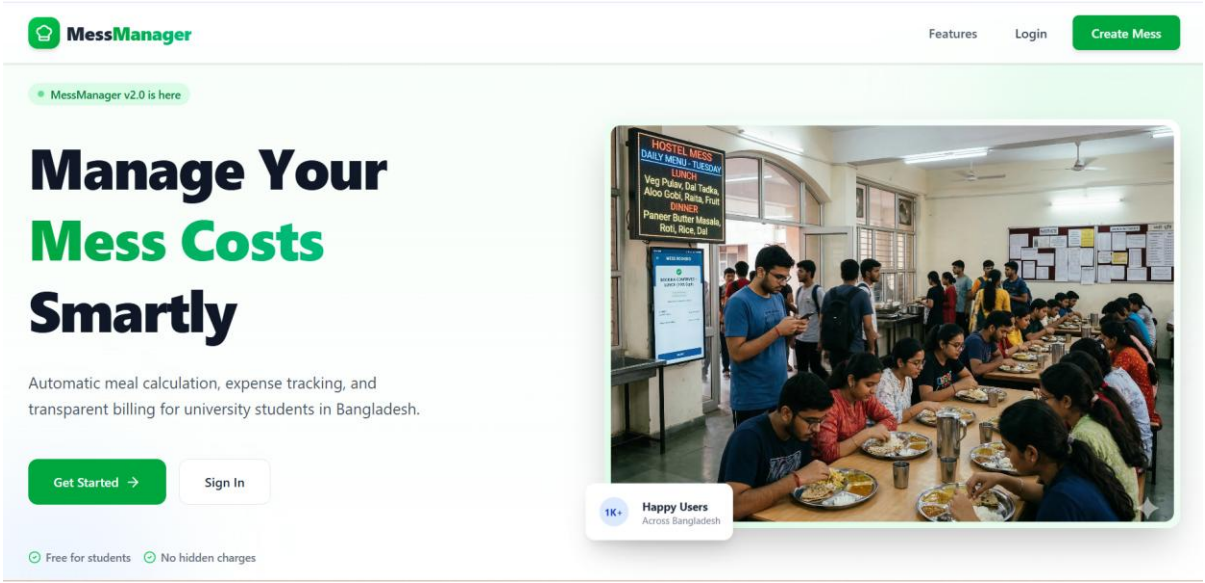


Figure 7.20 Home Page

7.4.2 Login Page

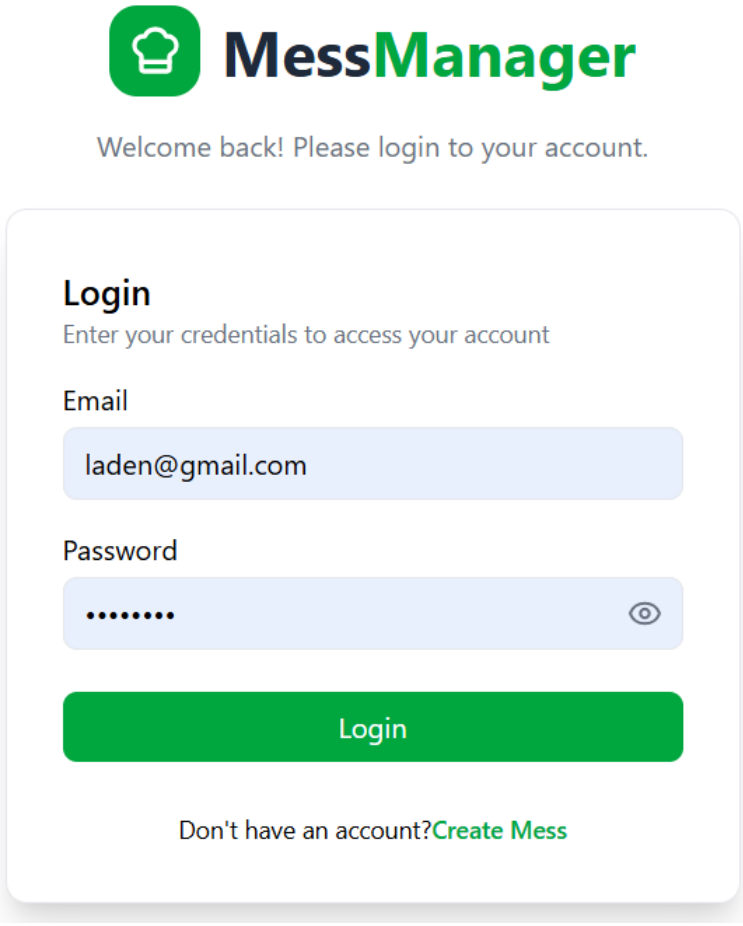


Figure 7.21 Login Page

7.4.3 Admin Interface

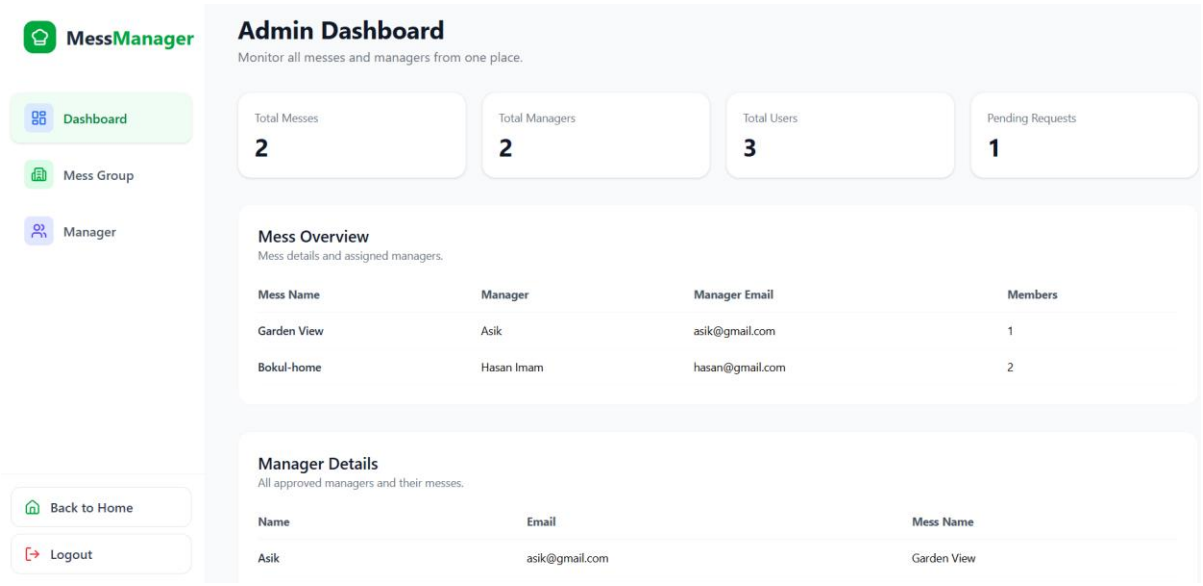


Figure 7.22 Admin Dashboard

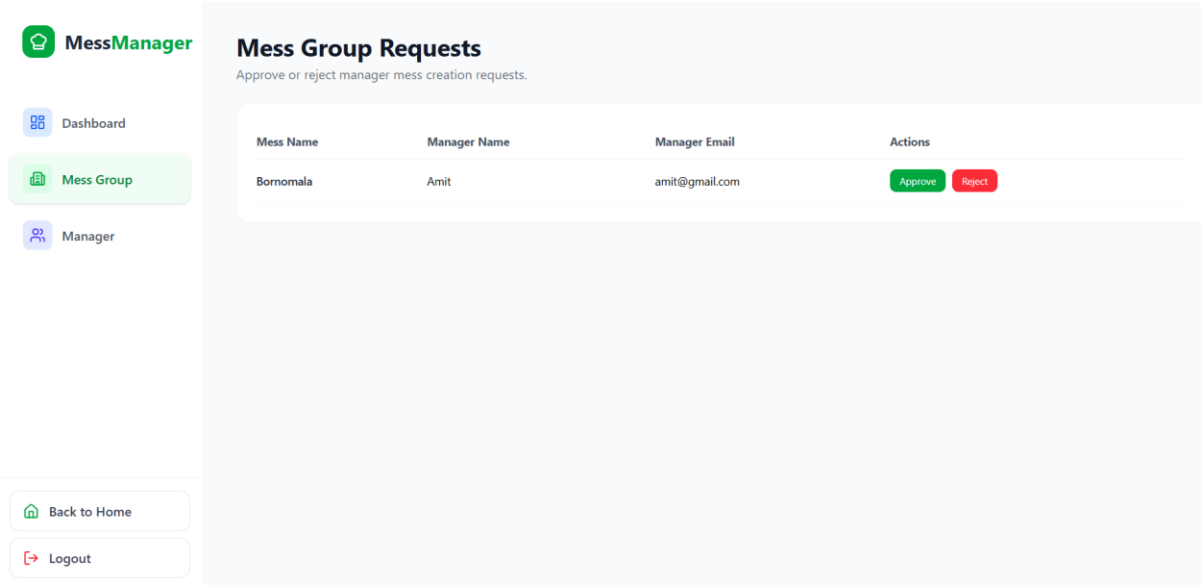


Figure 7.23 Mess Group Request

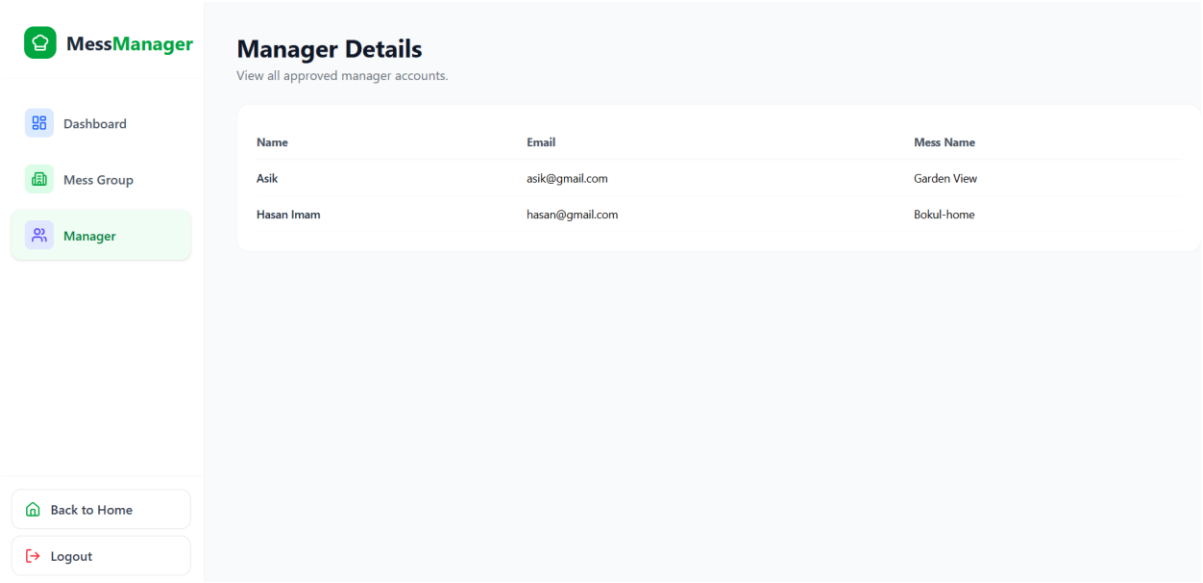


Figure 7.24 Manager Details

7.4.4 Manager Interface

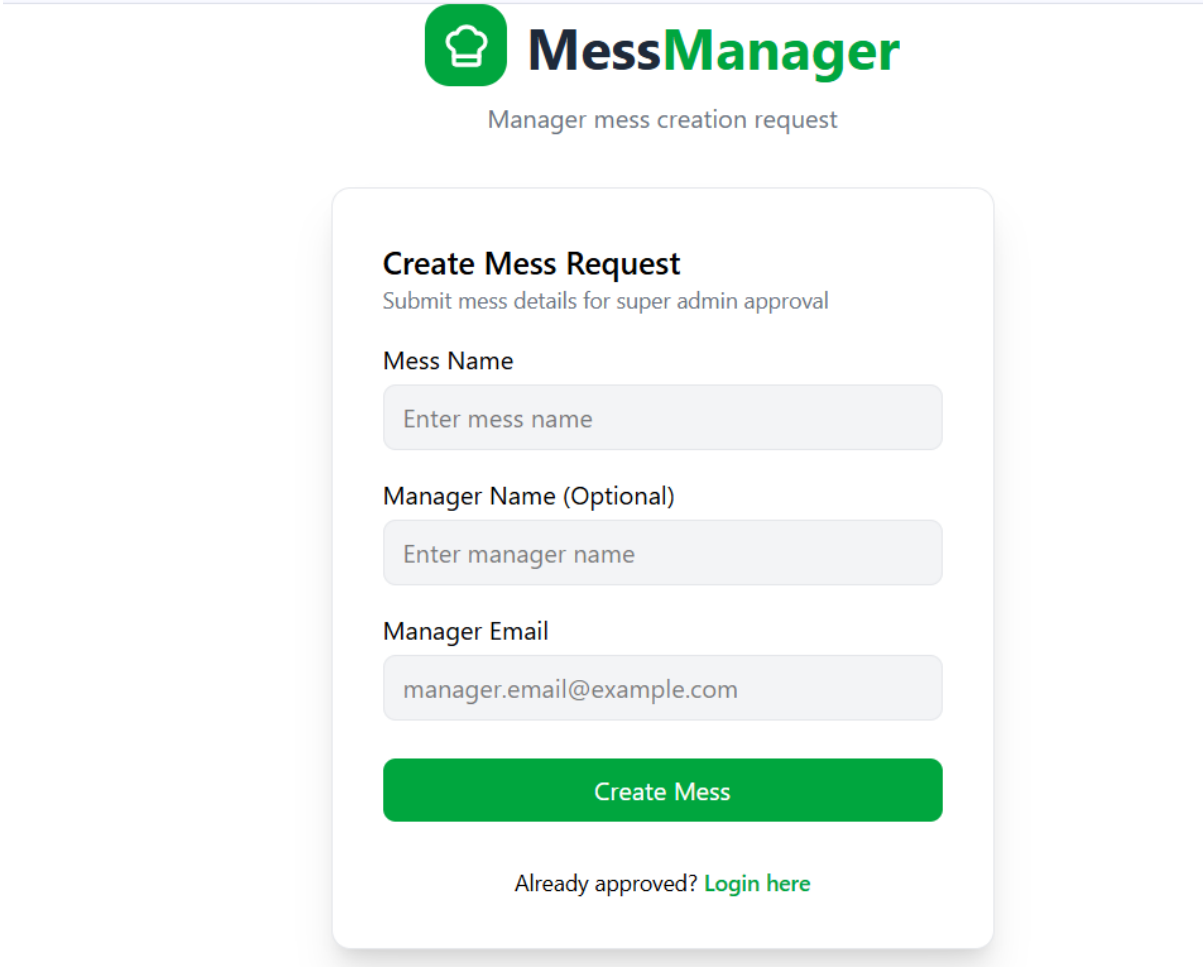


Figure 7.25 Create Mess Request

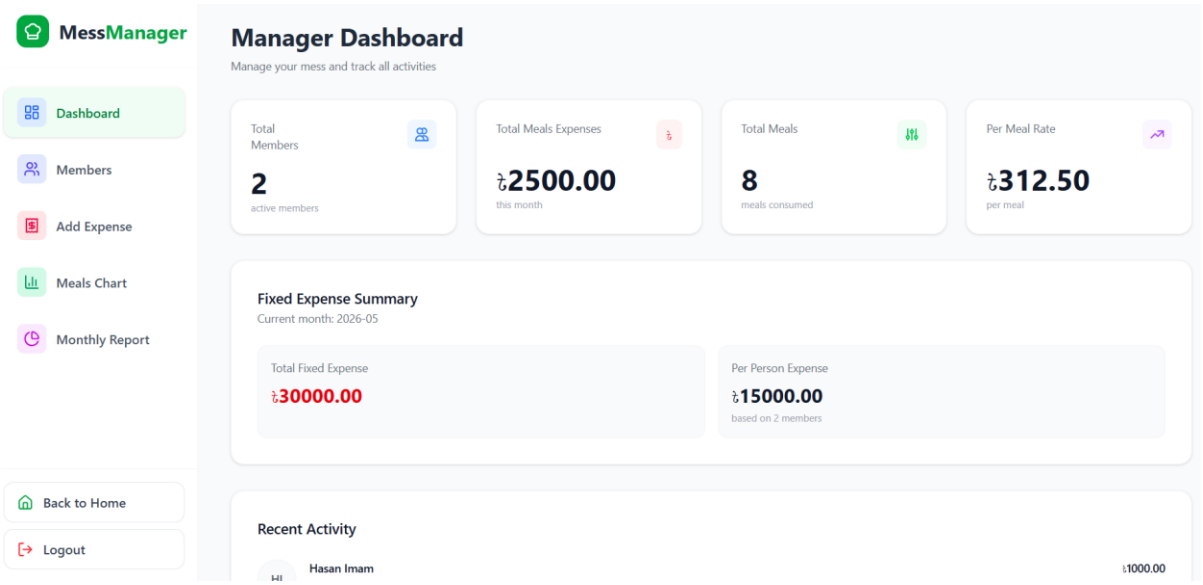


Figure 7.26 Manager Dashboard

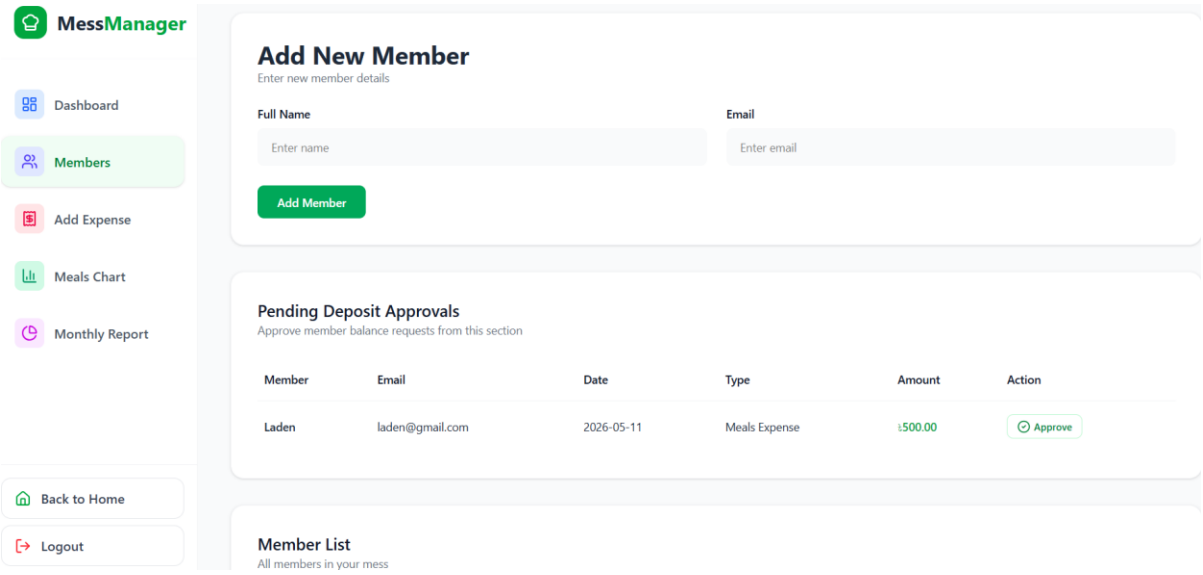


Figure 7.27 Add New Member

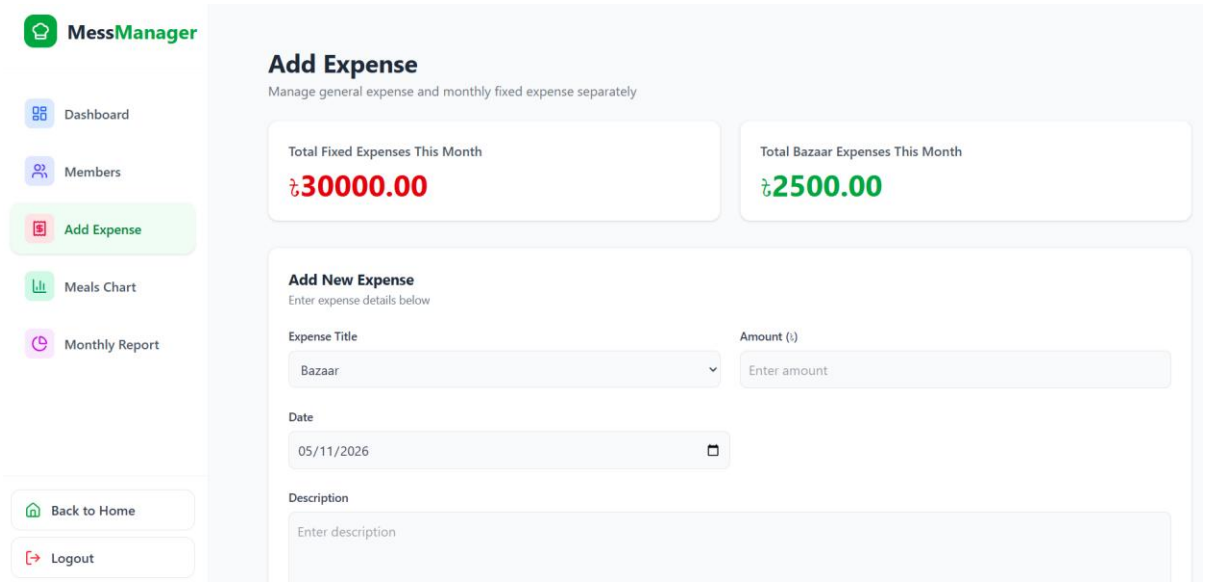


Figure 7.28 Add Expense

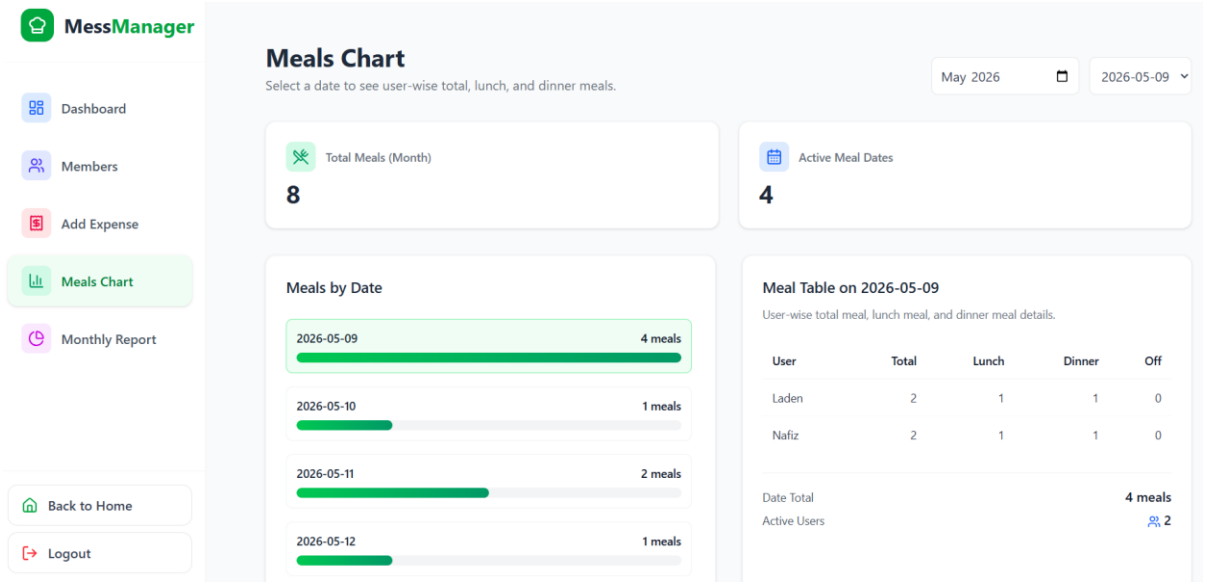


Figure 7.29 Meals Chart

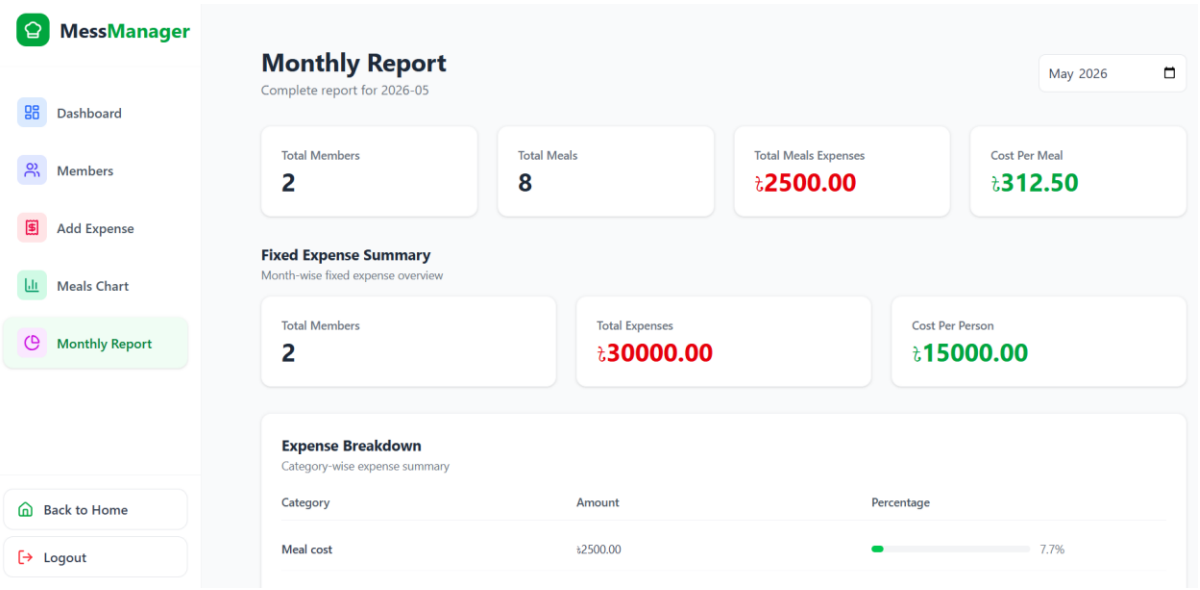


Figure 7.30 Monthly Report

7.4.5 User Interface

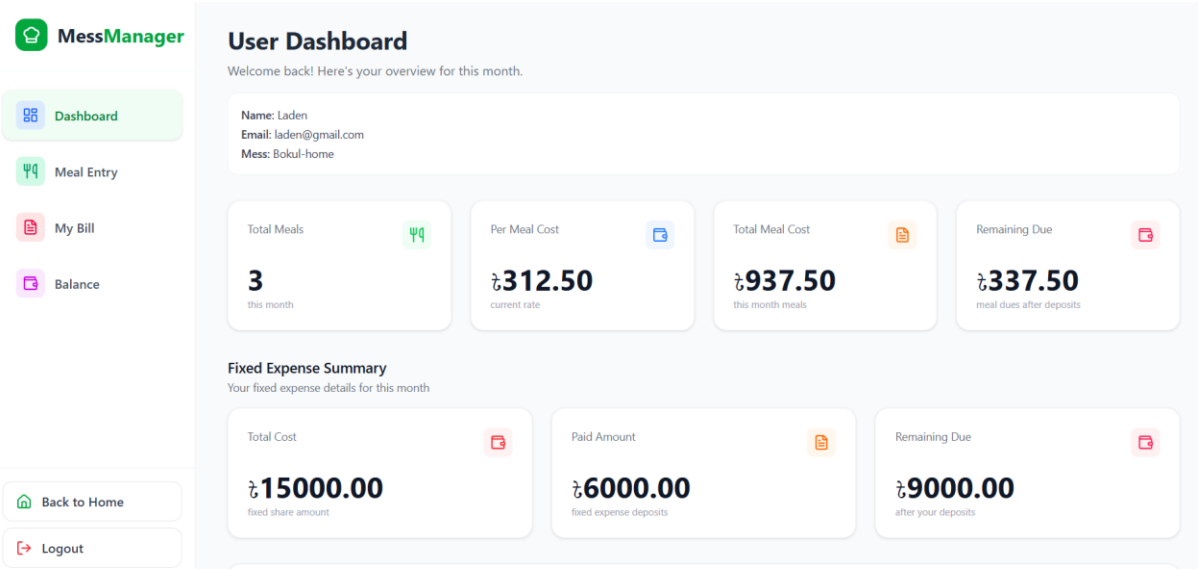


Figure 7.31 User Dashboard

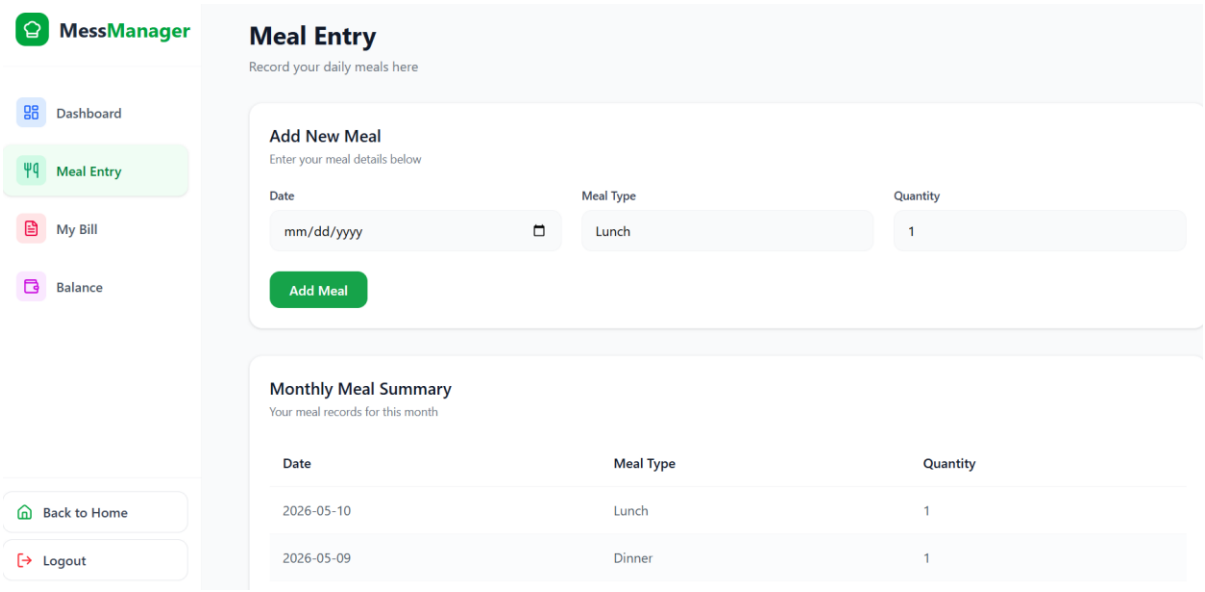


Figure 7.32 Meal Entry

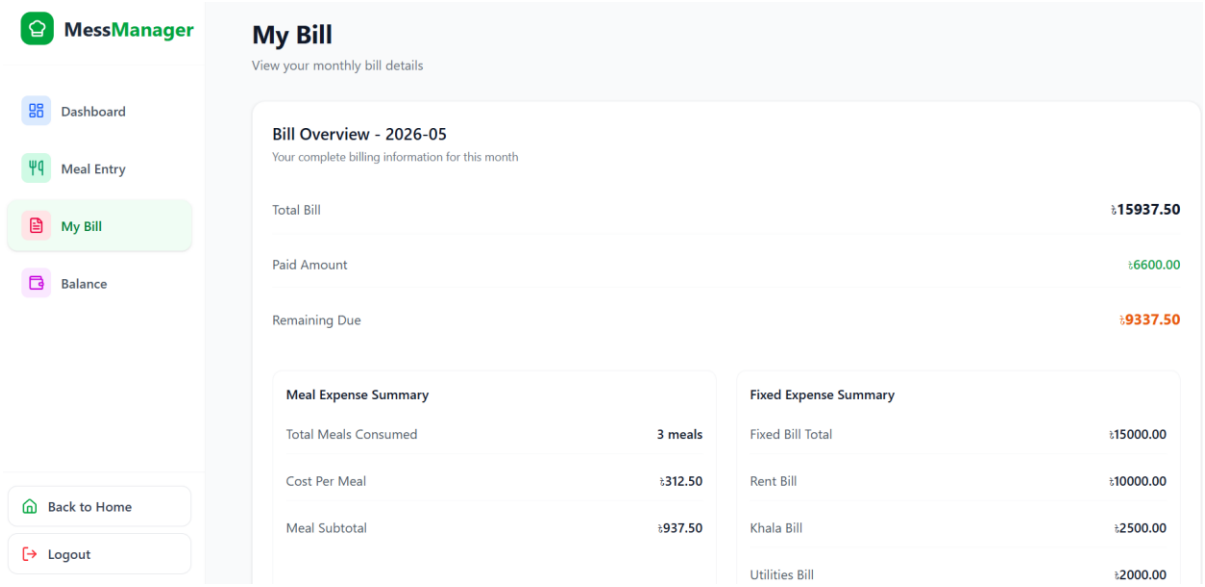


Figure 7.33 My Bill

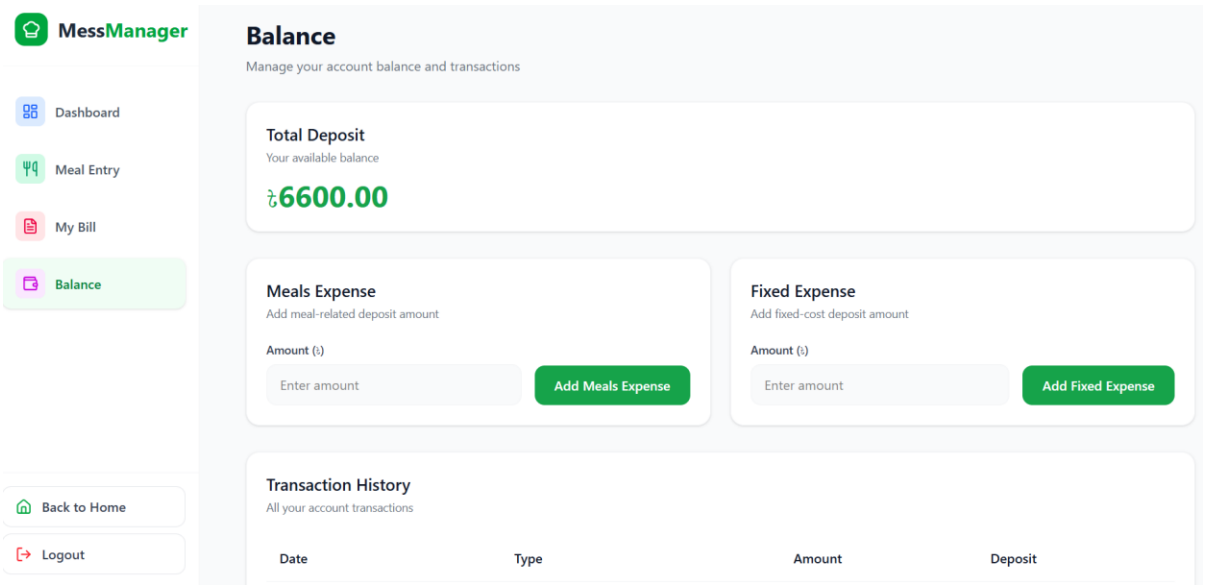


Figure 7.34 Balance

7.5 API Design Summary

The backend exposes several key RESTful endpoints to support frontend functionality.

Table 7.5: API Design Summary

Endpoint	Method	Role	Description
/api/register	POST	Public	Submit mess creation request.
/api/user/meals	POST	Member	Add or update a daily meal record.
/api/manager/fixed-costs	POST	Manager	Set rent and utilities for a specific month.
/api/user/bill/summary	GET	All	Fetch the current month's bill and meal rate.

7.6 Implementation Notes

The core business logic of the system resides in the Laravel Controllers. For example, the billing logic is encapsulated within the ExpenseController, which filters bazaar costs by their title to compute the dynamic meal rate. On the frontend, RTK Query is utilized for efficient data fetching and caching, (Meta Platforms, 2024) ensuring that the UI remains fast and responsive even with frequent data updates.

Chapter 8.
Testing

Testing is a critical phase in the development of MessManager to ensure that the software feature or function works correctly and reliably. This chapter outlines the strategies, environmental setup, and specific test cases used to verify that the system meets its functional requirements and provides an error-free experience for mess members and managers.

8.1 Test Strategy

The testing process followed a multi-layered approach to ensure both technical stability and a smooth user experience. (Society, 2014)

- **Unit Testing:** Focused on testing individual components, such as the billing logic in the ExpenseController and model relationships.
- **Integration Testing:** Verified that the React 19 frontend successfully communicates with the Laravel 10 REST API and that data is accurately stored in the MySQL database.
- **Manual Acceptance Testing:** Simulated real-world mess scenarios, such as multiple members logging meals and a manager approving deposits, to ensure the workflow aligns with user expectations.

8.2 Test Environment & Data

To maintain consistency, a controlled environment was established for all testing phases.

- **Software Stack:** Laravel 10 (PHP 8.1+), React 19, and MySQL.
- **Tools:** VS Code for debugging and Postman for API endpoint validation.
- **Test Data:** Automated database seeders were used to create test accounts for superAdmin, manager, and user roles to simulate a populated mess environment.

8.3 Test Case Tables

The following test cases document the execution steps, expected outcomes, and actual results for the core features of MessManager.

Table 8.1 Authentication and Role Access

Test Case ID	Module	Test Title	Expected Result	Actual Result	Status
TC-01	Auth	User Login Test	User is redirected to their specific dashboard based on role.	Logged in and redirected to Manager Dashboard.	Pass
TC-02	Auth	First Login Password Setup	User is forced to change their temporary password on first login.	"Change Password" prompt appeared and updated successfully.	Pass

Table 8.2 Core Functional Flows

Test Case ID	Module	Test Title	Expected Result	Actual Result	Status
TC-03	Meals	Add Meal Entry	System stores meal status and updates quantity to 0 for "Off".	Meal stored correctly with accurate quantities.	Pass
TC-04	Billing	Meal Rate Calculation	System correctly divides total bazaar costs by total meals.	Calculation matched manual verification exactly.	Pass
TC-05	Deposits	Deposit Approval	Manager approval creates a transaction and updates user balance.	Balance updated and transaction record created.	Pass

8.4 Test Results & Bug Log

The testing phase confirmed that the core modules—Authentication**,** Meal Tracking**,** and Billing Logic—function according to the defined specifications. Minor UI bugs were identified during manual acceptance testing, such as date formatting issues in the meal calendar, which were promptly resolved before the final deployment.

8.5 Performance & Security Checks

The system was verified to load dashboards within the target of 2-3 seconds. Security was confirmed through middleware checks, ensuring that a standard user cannot access manager routes or edit system-wide fixed costs.

.

Chapter 9.

Ethical Considerations & Sustainability

Fundamental ethical values served as the foundation for the creation of the MessManager system. Because the application handles both personal activity and sensitive financial data, it is built to benefit members and managers alike without compromising integrity or privacy. Transparency in how costs are allocated and the security of financial records are the pillars of this system's ethical framework.

9.1 Ethical Considerations

The development of MessManager prioritizes professional responsibility and fairness across all user roles.

9.1.1 Data Privacy and Security

Strong security measures are implemented to manage protected financial and personal information:

- **Authentication:** Bcrypt password hashing is used in conjunction with Laravel Sanctum to ensure secure user access.
- **Authorization:** Role-Based Access Control (RBAC) ensures that standard members cannot access manager settings or modify the bazaar expenses of others.
- **Confidentiality:** Individual deposit histories and bill summaries are restricted to the specific user and the mess manager to maintain financial privacy.
- **CSRF Protection:** All web forms include security tokens to prevent Cross-Site Request Forgery attacks.

9.1.2 Responsibility to Stakeholders

The system maintains a sense of accountability to its primary stakeholders:

- **To Mess Managers:** Providing accurate financial summaries and automated meal rate calculations for better decision-making.
- **To Members:** Ensuring that their digital records are accurate, transparent, and always accessible for verification.

9.2 Sustainability Through Software Engineering

The project contributes to environmental and economic sustainability by optimizing resource usage and eliminating physical waste.

9.2.1 Paperless Impact (Social & Environmental Sustainability)

MessManager aims to address real-world problems by contributing positively to the community without negative environmental impacts:

- **Reduction of Waste:** By digitizing meal registers and bazaar logs, the system eliminates the need for physical notebooks and paper-based accounting.
- **Long-term Accuracy:** Digital records prevent the loss of information that often occurs with physical documents, ensuring social sustainability through trust and transparency.

9.2.2 Economical Sustainability

Economic sustainability refers to developing a system that is cost-effective for both the developer and the end-users:

- **Open-Source Efficiency:** The use of Laravel, React, and MySQL reduces software licensing costs, making the platform affordable (Group, 2021) for student-managed messes.
- **Minimized Errors:** Automated calculations reduce financial discrepancies, leading to long-term economic stability within the mess group.

9.3 Professional Responsibility

Throughout the development of MessManager, professional standards were maintained by working honestly and with a mindset of continuous improvement. All third-party libraries and frameworks used, such as Redux Toolkit and Tailwind CSS, have been properly utilized according to their open-source licenses.

.

Chapter 10.

Conclusion

10.1 Conclusion

The primary goal of the MessManager project was to replace inefficient, manual paper-based mess tracking with a transparent and automated digital solution. By utilizing a modern Laravel 10 and React 19 tech stack, the system successfully addresses the core pain points of student shared living in Bangladesh:

- **Automation of Calculations:** The system now automatically computes the dynamic meal rate and splits monthly fixed costs equally among members, eliminating human calculation errors.
- **Transparency:** Role-based dashboards allow every member to view their real-time balance, deposit status, and total bill, fostering trust within the mess.
- **Operational Efficiency:** Managers can now oversee dozens of members, approve financial requests, and log expenses through a single unified interface, significantly reducing administrative overhead.

The successful implementation of this practicum project demonstrates that complex community-based financial operations can be streamlined through disciplined software engineering and user-centric design.

10.2 Limitations

While MessManager provides a robust foundation for mess operations, certain limitations were identified during the testing and evaluation phases that fall outside the current project scope:

- **Manual Expense Entry:** Managers must still manually type in bazaar expense titles and amounts, which can be time-consuming after large shopping trips.
- **Offline Access:** The application requires a stable internet connection to sync meal data and financial transactions with the MySQL database.
- **Currency Restrictions:** The current billing logic is hardcoded for Bangladeshi Taka (BDT) and does not yet support multi-currency or localized formatting for international users.

- **Mobile App Absence:** While the web interface is responsive, the lack of a dedicated native mobile application limits features like push notifications for meal reminders.

10.3 Future Work & Roadmap

The next phase of MessManager development focuses on increasing automation and improving the mobile user experience.

10.3.1 Character-Level OCR Integration

The most significant planned update is the integration of an OCR (Optical Character Recognition) module for bazaar receipts.

- **Transition Strategy:** Moving from page-level to character-level OCR for Bangla handwriting and printed text to ensure high accuracy for localized receipts.
- **Workflow:** Managers will be able to take a photo of a physical bazaar memo, and the system will automatically extract the date, title, and total amount to populate the expense log.
- **Technology:** Plans include utilizing a Convolutional Neural Network (CNN) classifier trained on datasets like Ekush or BanglaLekha-Isolated to better recognize handwritten Bangla digits and characters.

10.3.2 Functional Roadmap

- **Automated Notifications:** Implementing SMS or email alerts to remind members to log their "Off" meals before the daily deadline.
- **Digital Payment Integration:** Adding support for mobile financial services (MFS) like bKash or Nagad to allow members to pay their bills directly through the app.
- **Predictive Analytics:** Using historical data to forecast future bazaar costs, helping mess groups plan their budgets more effectively.

By executing this roadmap, MessManager will evolve from a record-keeping tool into a fully automated financial ecosystem, further enhancing the quality of life for students and residents in shared living environments.

References

Corporation, O., 2023. *MySQL 8.0 Reference Manual,* Austin, TX: s.n.

Group, T. P., 2021. *PHP 8.1 Manual.* [Online] Available at: https://www.php.net/docs.php

Meta Platforms, I., 2024. *React 19 Documentation.* [Online] Available at: https://react.dev/

Otwell, T., 2023. *Laravel 10.x Documentation.* [Online] Available at: https://laravel.com/docs/10.x

Society, I. C., 2014. *SWEBOK Guide: Guide to the Software Engineering Body of Knowledge.* [Online] Available at: https://www.computer.org/education/bodies-of-knowledge/software-engineering

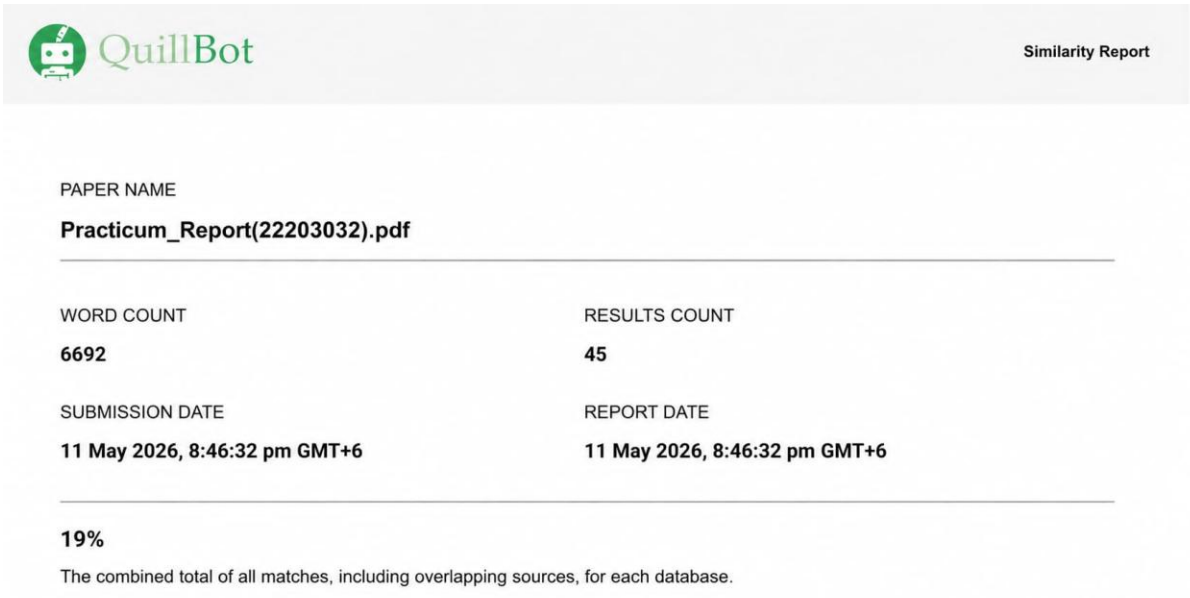


Figure 10.1 Plagiarism Report